

기존 rsync push 기반 파일 동기화 프로토콜에 uuid 및 동기화 메타 파일 정보 추가 방안

프로토콜 수정 방안

- Notion 노트 참고

- https://fixed-racer-fec.notion.site/24368fcfb34c80c38620ffb9df6cf155?source=copy_link

클라이언트의 DEVICE UUID 생성 설계

CMDEVICEUUIDMANAGER

- 신규 클래스

- 패키지: kr.ac.konkuk.ccslab.cm.util

- 불필요하게 과한 메소드 생략한 간단 버전 클래스 구현은 아래 링크 참조

- https://chatgpt.com/g/g-p-67ca5c79f79481919a611f83a51328d6-cm/shared/c/690bfaa9-8ef4-8322-8c7b-bf219477611f?owner_user_id=user-tJV1vRxu6MRhMLDCAxq3oCvv

- 전체 코드

```
package kr.ac.konkuk.ccslab.cm.util;
```

```
import kr.ac.konkuk.ccslab.cm.info.CMConfigurationManager; // ← CM 홈 제공자
import java.io.IOException;
import java.io.UncheckedIOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.nio.file.attribute.PosixFilePermission;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;
```

```
public final class CMDeviceUuidManager {
```

```
    private static final String[] REL_DIR = {".cm-settings","file-sync","client"};
    private static final String FILE_NAME = "device_uuid";
```

```
    private CMDeviceUuidManager() {} // 인스턴스화 금지
```

```
    /** 편의 함수: CM 홈(CMConfigurationManager) 기준으로 생성/획득 */
```

```
    public static UUID getOrCreateDeviceUuid() {
```

```
        String home = CMConfigurationManager.getInstance().getHomeDir(); // ← CM 홈
```

```

    return getOrCreateDeviceUuid(Paths.get(home));
}

/** 지정 baseDir 기준으로 생성/획득 (테스트/확장용) */
public static UUID getOrCreateDeviceUuid(Path baseDir) {
    Objects.requireNonNull(baseDir, "baseDir");
    final Path file = resolve(baseDir);

    try {
        Files.createDirectories(file.getParent());

        if (Files.isRegularFile(file)) {
            final String raw = Files.readString(file, StandardCharsets.UTF_8).trim();
            try {
                return UUID.fromString(raw);
            } catch (IllegalArgumentException bad) {
                // 손상 시 내부에서만 백업 처리
                backupCorrupted(file, raw, bad.getMessage());
                // 아래에서 새로 생성
            }
        }

        final UUID fresh = UUID.randomUUID();
        atomicWrite(file, fresh.toString() + "\n");
        secure600IfPossible(file);
        return fresh;
    } catch (IOException ioe) {
        throw new UncheckedIOException("device_uuid 준비 실패: " + file, ioe);
    }
}

/** 읽기(없거나 손상이면 null) */
public static UUID readDeviceUuid(Path baseDir) {
    final Path file = resolve(baseDir);
    try {
        if (!Files.isRegularFile(file)) return null;
        String raw = Files.readString(file, StandardCharsets.UTF_8).trim();
        return UUID.fromString(raw);
    } catch (Exception ignore) {
        return null;
    }
}

// ===== 내부 유틸 =====

private static Path resolve(Path baseDir) {
    Path dir = baseDir;
    for (String s : REL_DIR) dir = dir.resolve(s);
    return dir.resolve(FILE_NAME);
}

private static void atomicWrite(Path target, String content) throws IOException {
    final Path tmp = target.resolveSibling(target.getFileName() + ".tmp");
    Files.writeString(tmp, content, StandardCharsets.UTF_8,
        StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING);
    try {

```

```

        Files.move(tmp, target, StandardCopyOption.REPLACE_EXISTING,
StandardCopyOption.ATOMIC_MOVE);
    } catch (AtomicMoveNotSupportedException e) {
        Files.move(tmp, target, StandardCopyOption.REPLACE_EXISTING);
    }
}

private static void secure600IfPossible(Path p) {
    try {
        if (FileSystems.getDefault().supportedFileAttributeViews().contains("posix")) {
            Set<PosixFilePermission> perms = EnumSet.of(
                PosixFilePermission.OWNER_READ, PosixFilePermission.OWNER_WRITE);
            Files.setPosixFilePermissions(p, perms);
        }
    } catch (IOException ignore) { /* best-effort */ }
}

/** 손상 파일을 timestamp 로 백업하고 사유 기록 */
private static void backupCorrupted(Path file, String original, String reason) {
    try {
        String ts = DateTimeFormatter.ofPattern("yyyyMMdd-HH:mm:ss").format(LocalDateTime.now());
        Path bak = file.resolveSibling("device_uuid.corrupted." + ts);
        Files.move(file, bak, StandardCopyOption.REPLACE_EXISTING);

        String note = System.lineSeparator()
            + "# Corrupted device_uuid was backed up" + System.lineSeparator()
            + "# Time: " + ts + System.lineSeparator()
            + "# Reason: " + reason + System.lineSeparator()
            + "# Original: " + original + System.lineSeparator();

        Files.writeString(bak, note, StandardCharsets.UTF_8,
            StandardOpenOption.CREATE, StandardOpenOption.APPEND);
    } catch (IOException e) {
        System.err.println("CMDeviceUuidManager.backupCorrupted() failed: " + e);
    }
}
}

```

- 기존 CM 과 통합 방법

- 클라이언트 초기화 시점 (CMClientStub.startCM() 초반부)에 CM 홈을 기준으로 보장 값 확보

- ...

```
UUID deviceUuid = CMDeviceUuidManager.getOrCreateDeviceUuid();
```

CMFILESYNCHINFO

- 수정 방안

- 한 번 생성된 클라이언트의 device uuid 를 전역에서 재사용(로그인 전/후, 파일 동기화 모듈, 기타 서비스)

- 멤버 변수 추가

- Private UUID m_deviceUuid; // 4 client

- 멤버 메소드 추가

■ M_deviceUuid 의 setter/getter

● 수정 코드

```
// CMFileSyncInfo.java
package kr.ac.konkuk.ccslab.cm.info;

import java.util.UUID;

public class CMFileSyncInfo {
    // ... existing fields ...

    // [NEW]
    private UUID m_deviceUuid;

    // [NEW]
    public synchronized UUID getDeviceUuid() {
        return m_deviceUuid;
    }

    // [NEW]
    public synchronized void setDeviceUuid(UUID deviceUuid) {
        this.m_deviceUuid = deviceUuid;
    }

    // 디버그/덤프 시에는 전체 출력 대신 앞 8 글자만 노출 권장
}
```

CMCLIENTSTUB

● 메소드 수정

Public Boolean startCM()

■ 수정 방안

- CM 시작 직후, 서버 접속/로그인 이전에 device uuid 를 보장하고 전역에 캐시
- 주의: CM 홈디렉토리 값 준비되기 전이면 안되므로, CM 홈 확정된 후로 배치

■ 수정 코드

```
// CMClientStub.java
import kr.ac.konkuk.ccslab.cm.util.CMDeviceUuidManager;
import kr.ac.konkuk.ccslab.cm.info.CMFileSyncInfo;
import java.util.UUID;

public boolean startCM() {
    // 기존 초기화 로직...
    final boolean ret = /* 기존 start 로직 */ true;

    // [NEW] CM 홈 기준 device_uuid 확보(생성/복구 포함)
    final UUID devUuid = CMDeviceUuidManager.getOrCreateDeviceUuid();
    CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
    syncInfo.setDeviceUuid(devUuid);
}
```

```

if(CMInfo._CM_DEBUG) {
    System.out.println("[CM] deviceId=" + deviceId.toString().substring(0, 8) + "...");
}

// 이후 기존 네트워크 연결/로그인/세션 조인 등 진행
return ret;
}

```

서버에서 파일 동기화 중 클라이언트의 DEVICE UUID 관리 방안

- 관리 방안
 - 현재 파일 동기화 시작 사용자는 CMFileSyncGenerator 내에 userName 으로 저장
 - CMFileSyncGenerator 는 CMFileSyncInfo 에서 userName 을 key 로 하는 Map 으로 관리
 - 다중 로그인 지원하는 CM 에서는 user name 과 user uuid 로 사용자 특정
 - 파일 동기화 대상은 디바이스이며, 이는 user name 과 device uuid 로 특정
 - 아래 동기화 이벤트 수정에서 userName 은 initiatorName 으로 변경
 - initiatorDeviceUuid 는 CMFileSyncGenerator 내에 멤버 변수로 설정
 - CMFileSyncInfo 내의 generator map 의 key 는 userName 에서 (initiatorName, initiatorUuid)로 변경
 - Key 를 위한 record CMFileSyncSenderKey 정의
 - Initiator 가 파일 동기화를 시작하여 CMFileSyncGenerator 가 생성되는 시점에 initiatorName 과 함께 initiatorDeviceUuid 초기화

CMFILESYNCSENDERKEY

- 신규 record 역할
 - 서버(수신측)에서 클라이언트(송신측)의 파일 동기화 정보를 가진 CMFileSyncGenerator 값을 찾기 위한 key 역할
 - 모든 CM 이벤트의 기본 사용자 식별자는 (userName, deviceId)
 - 파일 동기화 기능은 이 기본 식별자가 아닌 (userName, deviceUuid)인 stateKey 를 사용하며, senderKey 는 stateKey 의 deviceUuid 를 조회하기 위한 상위 키로 사용
- 구현

```
public record CMFileSyncSenderKey(String initiatorName, String initiatorUuid) {
    @Override public String toString() { return initiatorName + "/" + initiatorUuid; }
}
```

CMFILESYNCGENERATOR

- 멤버 변수 변경 및 추가
 - Private String userName; -> private String initiatorName;
private UUID initiatorUuid; // 신규
private UUID initiatorDeviceUuid; // 신규
 - 생성자에 초기화 수정
public CMFileSyncGenerator(String initiatorName, UUID initiatorUuid, UUID initiatorDeviceUuid) {
 ...
 this.initiatorName = initiatorName;
 this.initiatorUuid = initiatorUuid;
 this.initiatorDeviceUuid = initiatorDeviceUuid;
 ...
}
 - Getter/setter 수정 및 메소드 호출 부분 점검

CMFILESYNCINFO

- 멤버 변수 변경
 - Private Map<String, CMFileSyncGenerator> syncGeneratorMap;
->
private Map<CMFileSyncSenderKey, CMFileSyncGenerator> syncGeneratorMap;
 - Getter 의 리턴 타입 변경
 - Getter 호출하는 부분 (12 군데) 수정
 - Getter 로 generator map 얻은 후에 Key 로 CMFileSyncSenderKey() 객체 만들어서 사용

기존 파일 동기화 이벤트 수정 설계

CMFILESYNCEVENT

- 수정 방안
 - 공통 헤더 필드 추가
 - initiatorName, initiatorUuid, initiatorDeviceUuid
 - 공통 헤더의 마샬링/언마샬링 추가
 - marshallCommonFSHeader/unmarshallCommonFSHeader

- 이벤트 ID 주석을 변경된 필드명에 맞게 업데이트

- 수정 코드

```
// CMFileSyncEvent.java
public abstract class CMFileSyncEvent extends CMEvent {

    // [NEW] FileSync 공통 헤더(이벤트 개시 주체)
    // 서버가 시작하는 이벤트도 고려하여 이름/UUID/디바이스 UUID 모두 문자열 직렬화
    private String m_initiatorName = "";
    private String m_initiatorUuid = ""; // UUID 문자열(표준 canonical form)
    private String m_initiatorDeviceUuid = ""; // UUID 문자열

    public CMFileSyncEvent() {
        m_nType = CMInfo.CM_FILE_SYNC_EVENT;
    }

    // [NEW] 공통 헤더 getter/setter
    public String getInitiatorName() { return m_initiatorName; }
    public void setInitiatorName(String v) { m_initiatorName = (v==null? "": v); }

    public String getInitiatorUuid() { return m_initiatorUuid; }
    public void setInitiatorUuid(String v) { m_initiatorUuid = (v==null? "": v); }

    public String getInitiatorDeviceUuid() { return m_initiatorDeviceUuid; }
    public void setInitiatorDeviceUuid(String v) { m_initiatorDeviceUuid = (v==null? "": v); }

    // -----
    // [중요] 하위 클래스 마샬/언마샬 표준화 훅(Hook) 메소드 추가
    // CMEvent 는 marshalBody()/unmarshalBody()를 abstract 로 강제.
    // CMFileSyncEvent 가 공통 헤더를 먼저 처리하고, 나머지는 하위로 위임.
    // -----
    @Override
    protected final void marshalBody() {
        marshalCommonFSHeader(); // [NEW] 공통 헤더 직렬화
        marshalBodyCore(); // [NEW] 하위 클래스 본문 직렬화
    }

    @Override
    protected final void unmarshalBody(ByteBuffer msg) {
        unmarshalCommonFSHeader(msg); // [NEW] 공통 헤더 역직렬화
        unmarshalBodyCore(msg); // [NEW] 하위 클래스 본문 역직렬화
    }

    // [NEW] 하위 클래스가 구현해야 할 코어 본문
    protected abstract void marshalBodyCore();
    protected abstract void unmarshalBodyCore(ByteBuffer msg);

    // [NEW] 공통 FS 헤더 직렬화/역직렬화
    protected void marshalCommonFSHeader() {
        // 버전 마커(앞으로 필드 확장 대비) - 1 바이트 버전
        m_bytes.put((byte)1); // version = 1
        putStringToByteBuffer(m_initiatorName); // String
        putStringToByteBuffer(m_initiatorUuid); // String(UUID)
        putStringToByteBuffer(m_initiatorDeviceUuid); // String(UUID)
    }
}
```

```

protected void unmarshallCommonFSHeader(ByteBuffer msg) {
    byte ver = msg.get(); // version
    // ver==1: 현재 포맷
    m_initiatorName = getStringFromByteBuffer(msg);
    m_initiatorUuid = getStringFromByteBuffer(msg);
    m_initiatorDeviceUuid = getStringFromByteBuffer(msg);
}

@Override
public String toString() {
    return getClass().getSimpleName()+"{" +
        "m_nType=" + m_nType +
        ", m_nID=" + m_nID +
        ", m_strSender=" + m_strSender + "\" +
        ", m_strReceiver=" + m_strReceiver + "\" +
        ", initiatorName=" + m_initiatorName + "\" +
        ", initiatorUuid=" + m_initiatorUuid + "\" +
        ", initiatorDeviceUuid=" + m_initiatorDeviceUuid + "\" +
        '}'";
}
}

```

- Version 마커

- 프로토콜 버전 확인을 위해 마샬링/언마샬링 단계에서만 사용

- 이벤트 ID 별 주석 업데이트 예시

```

/**
 * event ID of the CMFileSyncEventStartFileList class.
 */
// Fields: initiatorName, initiatorUuid, initiatorDeviceUuid, numTotalFiles
public static final int START_FILE_LIST = 10;

```

파일 동기화 하위 이벤트 클래스들 공통 수정

- 수정 방안

- marshallBody()/unmarshallBody() 대신 다음 메소드명으로 변경
 - marshallBodyCore()/unmarshallBodyCore()
 - marshallBody()/unmarshallBody()는 부모인 CMFileSyncEvent 에서 구현
 - 기존 userName 또는 requester 멤버 변수를 제거하고 부모에서 정의한 initiatorName 로 사용

- (예시 A) StartFileList (기존: userName -> initiatorName)

```

// CMFileSyncEventStartFileList.java
public class CMFileSyncEventStartFileList extends CMFileSyncEvent {
    // [REMOVE] private String userName;
    private int numTotalFiles;

    // [OPTIONAL SHIM] 하위 호환용
    @Deprecated

```



```

public String getUserName() { return getInitiatorName(); }
@Deprecated
public void setUserName(String name) { setInitiatorName(name); }

public int getNumTotalFiles() { return numTotalFiles; }
public void setNumTotalFiles(int n) { numTotalFiles = n; }

@Override
protected void marshallBodyCore() {
    // 공통 헤더는 이미 부모가 처리
    m_bytes.putInt(numTotalFiles);
}

@Override
protected void unmarshallBodyCore(ByteBuffer msg) {
    numTotalFiles = msg.getInt();
}

@Override
public String toString() {
    return super.toString() + ", numTotalFiles=" + numTotalFiles + "";
}
}

```

- (예시 B) OnlineModeList (기존: requester -> initiatorName)

```

// CMFileSyncEventOnlineModeList.java
public class CMFileSyncEventOnlineModeList extends CMFileSyncEvent {
    // [REMOVE] private String requester;
    private List<Path> relativePathList = new ArrayList<>();

    @Deprecated
    public String getRequester() { return getInitiatorName(); }
    @Deprecated
    public void setRequester(String r) { setInitiatorName(r); }

    public List<Path> getRelativePathList() { return relativePathList; }
    public void setRelativePathList(List<Path> list) { relativePathList = list; }

    @Override
    protected void marshallBodyCore() {
        // 리스트 크기
        m_bytes.putInt(relativePathList.size());
        // 리스트 내용
        for (Path p : relativePathList) {
            putStringToByteBuffer(p.toString());
        }
    }

    @Override
    protected void unmarshallBodyCore(ByteBuffer msg) {
        int listSize = msg.getInt();
        relativePathList = new ArrayList<>(listSize);
        for (int i=0; i<listSize; i++) {
            relativePathList.add(Paths.get(getStringFromByteBuffer(msg)));
        }
    }
}

```

}

CMFILESYNCEVENTREQUESTNEWFILES 수정

- 멤버 변수 삭제
 - 이벤트 필드에서 String requesterName 삭제
- 멤버 메소드 수정
 - requesterName 의 getter/setter 삭제
 - getByteNum(), marshallBodyCore(), unmarshallBodyCore(), toString(), equals(), hashCode() 내에 requesterName 관련 삭제

CMFILESYNCEVENTCOMPLETEFILESYNC 수정

- 멤버 변수 추가
 - 이벤트 필드로 long cursor 추가
 - 용도: 동기화 세션이 완료되었을 때, 동기화 메타 정보로 서버가 클라이언트로 업데이트된 lastChangeId 정보 전달 (메모리 및 파일 업데이트용)
- 멤버 메소드 수정
 - Getter/setter 추가
 - getByteNum(), marshallBodyCore(), unmarshallBodyCore(), toString(), equals(), hashCode() 내에 cursor 관련 추가

CMFILESYNCEVENTCOMPLETENEWFILE 수정

- 멤버 변수 추가
 - 이벤트 필드로 long cursor 추가
 - 용도: 신규 파일이 추가되었을 때, 동기화 메타 정보로 서버가 클라이언트로 업데이트된 lastChangeId 정보 전달 (메모리 업데이트용)
- 멤버 메소드 수정
 - Getter/setter 추가
 - getByteNum(), marhsallBodyCore(), unmarshallBodyCore(), toString(), equals(), hashCode() 내에 cursor 관련 추가

CMFILESYNCEVENTCOMPLETEUPDATEFILE 수정

- 멤버 변수 추가
 - 이벤트 필드로 long cursor 추가
 - 용도: 기존 파일이 업데이트되었을 때, 동기화 메타 정보로 서버가 클라이언트로 업데이트된 lastChangeId 정보 전달 (메모리 업데이트용)
- 멤버 메소드 수정
 - Getter/setter 추가
 - getByteNum(), marhsallBodyCore(), unmarshallBodyCore(), toString(), equals(), hashCode() 내에 cursor 관련 추가

CMFILESYNCEVENTCOMPLETEDELETEFILES 추가

- 이벤트 클래스 역할
 - 서버가 클라이언트로 파일 삭제가 완료되었음을 알리기 위한 용도
 - 신규 파일이 추가되거나 기존 파일이 업데이트되었을 때는 이에 대한 완료 이벤트를 클라이언트로 전송하는데 삭제에 대해서는 별도로 알리는 이벤트가 아직 없어서 프로토콜 일관성 차원으로, 또한 다중 디바이스 동기화 프로토콜 대비용으로 추가
- 구현 아이디어
 - 개별 파일에 대해 이벤트를 보내지 않고, 파일 모드 변경 이벤트와 유사하게 이벤트 최대 크기 내에서 삭제된 파일 리스트를 포함
 - 이 이벤트를 생성하는 코드에서, 파일 모드 리스트 이벤트처럼 이벤트 최대 크기를 벗어나지 않도록 리스트를 구성해야 함
 - 클라이언트의 메모리 cursor 업데이트 위해 cursor 값 추가
- 멤버 변수
 - Private List<Path> deletedPathList;
 - Long cursor;
- 멤버 메소드
 - Getter/setter 추가
 - 생성자, getByteNum(), marshallBodyCore(), unmarshallBodyCore(), toString(), equals(), hashCode()

동기화 시작에서 완료까지의 이벤트 송수신 처리 수정

- 수정 방안
 - 파일 동기화 프로토콜별로 이벤트 흐름 순서대로 수정 진행
 - 파일 동기화 시작
 - 파일 동기화 중지 -> 이벤트 전송 안함
 - 온라인 모드로 변경
 - 로컬 모드로 변경
 - 동기화 작업별 송수신 이벤트 현황 정리
 - 참고: <https://chatgpt.com/share/68b2809c-b7ec-800f-ba37-366e65b3f589>

START_FILE_LIST 송신

- CMFileSyncManager

```
public Boolean startFileSync(CMFileSyncMode fileSyncMode) ->
public synchronized Boolean sync() ->
private Boolean sendFileList()
```

 - 메소드 역할
 - 클라이언트가 파일 리스트 전송 시작 알림
 - 수정 방안
 - 이벤트 필드에 userName 대신 initiatorName 으로 변경
 - 이벤트 필드에 initiatorUuid, initiatorDeviceUuid 추가
 - 수정 구현
 - CMInteractionInfo, CMFileSyncInfo 구하기

```
CMInteractionInfo interInfo = CMInteractionInfo.getInstance();
CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
// 이후 interInfo, syncInfo 사용하는 곳에 적용
```
 - 변수명 변경

```
String userName -> String initiatorName;
String serverName -> String receiverName;
...
```
 - 변수 추가

```
...
```

```

UUID initiatorUuid = interInfo.getMyself().getUuid();
UUID initiatorDeviceUuid = syncInfo.getDeviceUuid();
UUID receiverUuid = null;
...

```

- // get my name 코멘트부터 // get path list 코멘트 전라인까지 필드 설정 부분을 다음으로 수정

```

...
// get initiator name
initiatorName = interInfo.getMyself().getName();
// get default server name (양방향 동기화 때 수정 필요)
receiverName = interInfo.getDefaultServerInfo().getServerName();
// set common initiator, initiator uuid, initiator device uuid
fse.setInitiatorName(initiatorName);
fse.setInitiatorUuid(initiatorUuid);
fse.setInitiatorDeviceUuid(initiatorDeviceUuid);
// sender, receiver 설정은 삭제 (unicastEvent()에서 일괄 설정 예정)

```

- // get path list 아래 설정 부분 기존 코드 동일
- ...
- // send the event 다음 라인 unicast 호출을 다음으로 수정
Boolean sendResult = CMEventManager.unicastEvent(fse, receiverName, **receiverUuid**);
- 나머지 기존 코드 동일
- ...

START_FILE_LIST 수신 및 START_FILE_LIST_ACK 송신

- CMFileSyncEventHandler

private Boolean processSTART_FILE_LIST(CMFileSyncEvent fse)

■ 메소드 역할

- 파일 목록 전송 시작에 대한 ack 을 initiator 에게 전송

■ 수정 방안

- 이벤트 필드에 공통 initiatorName, initiatorUuid, initiatorDeviceUuid 추가
- 이벤트 필드에 Sender, receiver, userName 설정은 삭제

■ 수정 구현

- userName 변수 라인 다음으로 수정
String initiatorName = fse_sfl.getInitiatorName(); // 다른 이 변수 사용 부분도 수정
- initiator uuid 변수 추가
UUID initiatorUuid = fse_sfl.getInitiatorUuid();

- CMFileSyncManager 구하기 다음으로 수정
 CMFileSyncManager fsManager =
 CMInfo.getInstance().getServiceManager(CMFileSyncManager.class);
- Server sync home 관련 부분 일단 그대로
 // 추후 양방향 파일 동기화시에 server 명칭보다 receiver 명칭으로 수정 필요
 ...
- // create the ack event 아래 이벤트 설정을 다음으로 수정
 CMFileSyncEventStartFileListAck ackFse = new CMFileSyncEventStartFileListAck();
 // 공통 필드 설정
 ackFse.setInitiatorName(initiatorName);
 ackFse.setInitiatorUuid(initiatorUuid());
 ackFse.setInitiatorDeviceUuid(fse_fsl.getInitiatorDeviceUuid());
 // 나머지 필드 설정 (num total files, return code)
 ackFse.setNumTotalFiles(fse_fsl.getNumTotalFiles());
 ackFse.setReturnCode(1);
- // send the ack event to the client 아래 라인 다음으로 수정
 return CMEventManager.unicastEvent(ackFse, initiatorName, initiatorUuid);

START_FILE_LIST_ACK 수신 및 FILE_ENTRIES 송신

- CMFileSyncEventFileEntries
 - 멤버 변수명 (이벤트 필드명) 변경
 private List<CMFileSyncEntry> clientPathEntryList;
 ->
 private List<CMFileSyncEntry> **initiatorPathEntryList**;
 - 생성자 초기화, getter/setter 이름 수정
 - getByteNum(), marshallBodyCore(), unmarshallBodyCore() 내 변경된 변수명 적용
- CMFileSyncEventHandler
private Boolean processSTART_FILE_LIST_ACK(CMFileSyncEvent fse)
 - 메소드 역할
 - 파일 목록들 담은 이벤트 (FILE_ENTRIES) 전송
 - 한 이벤트에 담은 목록 만들어 설정하는건 setNumFilesAndEntryList() 메소드에서 처리
 - 목록이 긴 경우에는 이 이벤트의 ack 수신 때 다음 목록 이어서 전송
 - 수정 방안
 - 이벤트 필드에 공통 initiatorName, initiatorUuid, initiatorDeviceUuid 추가
 - 이벤트 필드에 Sender, receiver, userName 설정은 삭제

■ 수정 구현

- String server = ... 변수 다음으로 수정
String receiver = fse_sfla.getSender();
String receiverUuid = fse_sfla.getSenderUuid();
- // create a FILE_ENTRIES event 아래를 다음으로 수정
CMFileSyncEventFileEntries newfse = new CMFileSyncEventFileEntries();
// 공통 필드 설정
newfse.setInitiatorName(fse_sfla.getInitiatorName());
newfse.setInitiatorUuid(fse_sfla.getInitiatorUuid());
newfse.setInitiatorDeviceUuid(fse_sfla.getInitiatorDeviceUuid());
// 나머지 필드 설정
newfse.setNumFilesCompleted(0);
// set numFiles and fileEntryList
setNumFilesAndEntryList(newfse, 0); // 아래에서 메소드 수정
- 마지막 return 문 다음으로 수정
return CMEventManager.unicastEvent(newfse, receiver, receiverUuid);

Private CMFileSyncEvent setNumFilesAndEntryList(CMFileSyncEventFileEntries newfse, int startListIndex)

■ 메소드 역할

- 파일 동기화 FILE_ENTRIES 이벤트에서 이번에 전송할 엔트리 개수와 엔트리 리스트 필드를 설정하는 메소드
- CM 이벤트 최대 길이 내에서 파일 엔트리 리스트와 개수 구함

■ 수정 방안

- CMInfo, CMFileSyncinfo reference 구하는 방법만 수정
- Newfse 이벤트에 엔트리 리스트 설정 메소드 이름 수정
newfse.setInitiatorPathEntryList(fileEntryList);

FILE_ENTRIES 수신 및 FILE_ENTRIES_ACK 송신

- CMFileSyncInfo (이벤트 수신 처리를 위한 info 내 정보 수정)

■ 멤버 변수 변경

- Private Map<String, List<CMFileSyncEntry>> clientPathEntryListMap;
->
private Map<CMFileSyncStateKey, List<CMFileSyncEntry>> initiatorPathEntryListMap;
- Private Map<String, List<Path>> basisFileListMap;
->
private Map<CMFileSyncStateKey, List<Path>> basisFileListMap;

- 멤버 메소드 변경
 - 위 수정 변수의 getter 수정


```
Map<CMFileSyncStateKey, List<CMFileSyncEntry>> getInitiatorPathEntryListMap();
Map<CMFileSyncStateKey, List<Path>> getBasisFileListMap();
```
- Getter 호출부분 수정
 - 기존 `userName` 만을 키로 `value` 를 검색하는 것이 아니고, 같은 사용자의 여러 디바이스가 가능하므로 `(userName, userDeviceUuid)` 값(record `CMFileSyncStateKey` 객체)을 key 로 value 검색해야 함
 - `initiatorDeviceUuid` 를 찾아야 할 때는, `CMFileSyncSenderKey(initiatorName, initiatorUuid)`를 key 로 `CMFileSyncGenerator` 객체 검색해서 `getInitiatorDeviceUuid()`로 구하기
- `CMFileSyncEventHandler`

```
private Boolean processFILE_ENTRIES(CMFileSyncEvent fse)
```

 - 메소드 역할
 - 송신측(클라이언트)이 보낸 파일 목록 엔트리들을 처리하고 ack 이벤트 전송
 - Ack 이벤트 전송 후에 다음 파일 목록으로 같은 이벤트를 반복 수신할 수 있음
 - 수정 방안
 - `clientPathEntryListMap` 사용 부분 수정
 - `getInitiatorPathEntryListMap()`으로 getter 변경
 - key 로 record `CMFileSyncStateKey (initiatorName, initiatorDeviceUuid)` 사용
 - Ack 이벤트 필드에 공통 `initiatorName, initiatorUuid, initiatorDeviceUuid` 추가
 - Ack 이벤트 필드에 `Sender, receiver, userName` 설정은 삭제
 - `basisFileListMap` 사용 부분 수정 (not yet 언제 하는지 추적 필요)
 - 수정 구현
 - `CMFileSyncInfo` 구하기


```
CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
...
```
 - `userName` 변수 할당 다음으로 수정


```
...
String initiatorName = fse_fe.getInitiatorName();
UUID initiatorUuid = fse_fe.getInitiatorUuid();
UUID initiatorDeviceUuid = fse_fe.getInitiatorDeviceUuid();
```


- `// if 0, the entry list is null in the event` 아래 `if` 문 다음으로 수정

```

if( numFiles > 0 ) {
    // set or add the entry list of the event to the entry map
    CMFileSyncStateKey key = new record CMFileSyncStateKey(initiatorName,
initiatorDeviceUuid);
    List<CMFileSyncEntry> entryList = syncInfo.getInitiatorPathEntryListMap().get(key);
    if( entryList == null ) {
        // set the new entry list to the Map
        syncInfo.getInitiatorPathEntryListMap().put(key, fse_fe.getInitiatorPathEntryList());
        // set the number of completed files
        numFilesCompleted = numFiles;
    }
    else {
        ... (getClientPathEntryList() 대신 getInitiatorPathEntryList()로 메소드명만 변경)
    }
}
}
...

```
- `// create FILE_ENTRIES_ACK event` 아래 이벤트 설정을 다음으로 수정

```

CMFileSyncEventFileEntriesAck fseAck = new CMFileSyncEventFileEntriesAck();
// 공통 필드 설정
fseAck.setInitiatorName( initiatorName );
fseAck.setInitiatorUuid( initiatorUuid );
fseAck.setInitiatorDeviceUuid( initiatorDeviceUuid );
// 나머지 필드 설정
fseAck.setNumFilesCompleted(numFilesCompleted); // updated
fseAck.setNumFiles(numFiles);
fseAck.setReturnCode(returnCode);

```
- `// send the ack event` 아래 `return` 문 다음으로 수정

```

return CMEEventManager.unicastEvent(fseAck, initiatorName, initiatorUuid);

```

FILE_ENTREIS_ACK 수신 및 END_FILE_LIST 송신

- `CMFileSyncEventHandler`
private Boolean processFILE_ENTRIES_ACK(CMFileSyncEvent fse)
 - 메소드 역할
 - 남아 있는 파일 엔트리 리스트를 계속 송신하거나 파일 엔트리 리스트 마지막이라는 이벤트를 수신측으로 송신
 - 두 작업 모두 아래 별도 메소드에서 처리
 - 수정 방안
 - 이 메소드 자체는 수정 사항 없음

Private Boolean sendNextFileEntries(CMFileSyncEventFileEntriesAck fse)

- 메소드 역할

- 남아 있는 파일 엔트리 리스트를 수신측으로 송신
- 수정 방안
 - 이벤트 필드에 공통 initiatorName, initiatorUuid, initiatorDeviceUuid 추가
 - 이벤트 필드에 Sender, receiver, userName 설정은 삭제
- 수정 구현
 - CMFileSyncInfo 구하기 삭제
 - ...
 - FILE_ENTRIES 이벤트 설정 부분 다음으로 수정


```
CMFileSyncEventFileEntries newfse = new CMFileSyncEventFileEntries();
// 공통 필드 설정
newfse.setInitiatorName( fse.getInitiatorName() );
newfse.setInitiatorUuid( fse.getInitiatorUuid() );
newfse.setInitiatorDeviceUuid( fse.getInitiatorDeviceUuid() );
// 나머지 필드 설정
newfse.setNumFilesCompleted( fse.getNumFilesCompleted() );
// set numFiles and fileEntryList
...
```
 - // send FILE_ENTRIES event 아래 return 문 다음으로 수정


```
return CMEventManager.unicast( newfse, fse.getSender(), fse.getSenderUuid() );
```

Private Boolean sendEND_FILE_LIST(CMFileSyncEventFileEntriesAck fse)

- 메소드 역할
 - 파일 엔트리 목록 전송이 완료되었음을 알리는 이벤트를 수신측에 전송
- 수정 방안
 - 이벤트 필드에 공통 initiatorName, initiatorUuid, initiatorDeviceUuid 추가
 - 이벤트 필드에 Sender, receiver, userName 설정은 삭제
- 수정 구현
 - // create an END_FILE_LIST event 아래 이벤트 설정 부분 다음으로 수정


```
CMFileSyncEventEndFileList newfse = new CMFileSyncEventEndFileList();
// 공통 필드 설정
newfse.setInitiatorName( fse.getInitiatorName() );
newfse.setInitiatorUuid( fse.getInitiatorUuid() );
newfse.setInitiatorDeviceUuid( fse.getInitiatorDeviceUuid() );
// 나머지 필드 설정
newfse.setNumFilesCompleted( fse.getNumFilesCompleted() );
```

- Return 문 다음으로 수정

```
// send the event to the sync receiver
return CMEventManager.unicastEvent( newfse, fse.getSender(), fse.getSenderUuid() );
```

END_FILE_LIST 수신 및 END_FILE_LIST_ACK 송신

- CMFileSyncEventHandler

private Boolean processEND_FILE_LIST(CMFileSyncEvent fse)

■ 메소드 역할

- 클라이언트(송신측)이 보낸 파일 목록 전송 완료 이벤트를 처리
- 파일 목록 개수 확인 등을 통해 return code 결정하여 ack 이벤트 전송

■ 수정 방안

- 이벤트 필드에 공통 initiatorName, initiatorUuid, initiatorDeviceUuid 추가
- 이벤트 필드에 Sender, receiver, userName 설정은 삭제
- CMFileSyncGenerator 객체 생성시 userName 이 아닌 initiatorName, initiatorUuid, initiatorDeviceUuid 정보 전달하고 CMFileSyncInfo 의 generatorMap 에 저장 (key 는 CMFileSyncSenderKey)

■ 수정 구현

- CMFileSyncInfo 변수 선언

```
CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
...
```
- // check the elements of file entry list 까지 기존 코드 동일

```
...
```
- String userName = ... 라인 다음으로 수정

```
String initiatorName = fse_efl.getInitiatorName();
UUID initiatorUuid = fse_efl.getInitiatorUuid();
UUID initiatorDeviceUuid = fse_efl.getInitiatorDeviceUuid();

int numFilesCompleted = ... // 기존 코드 동일
List<CMFileSyncEntry> fileEntryList =
syncInfo.getInitiatorPathEntryListMap().get(initiatorName, initiatorDeviceUuid);
... // 이하 기존 코드 동일
```
- // create an END_FILE_LIST_ACK event 아래 이벤트 설정 부분 다음으로 수정

```
CMFileSyncEventEndFileListAck fseAck = new CMFileSyncEventEndFileListAck();
// 공통 필드 설정
fseAck.setInitiatorName( initiatorName );
fseAck.setInitiatorUuid( initiatorUuid );
fseAck.setInitiatorDeviceUuid( initiatorDeviceUuid );
```

// 나머지 필드 설정

fseAck.setNumFilesCompleted(numFilesCompleted);

fseAck.setReturnCode(returnCode);

- // send the ack event 아래 unicast() 호출 다음으로 수정
Boolean result = CMEventManager.unicastEvent(fseAck, initiatorName, initiatorUuid);
...
- // start CMFileSyncGeneratorTask 아래 객체 생성 라인 다음으로 수정
CMFileSyncGenerator fileSyncGenerator = new CMFileSyncGenerator(initiatorName, initiatorUuid, initiatorDeviceUuid);
...
- // set the generator in the CMFileSyncInfo 아래 라인 다음으로 수정
CMFileSyncSenderKey key = new CMFileSyncSenderKey(initiatorName, initiatorUuid);
syncInfo.getSyncGeneratorMap().put(key, fileSyncGenerator);
- Return true;

END_FILE_LIST_ACK 수신

- CMFileSyncEventHandler
private boolean processEND_FILE_LIST_ACK(CMFileSyncEvent fse)
 - 메소드 역할
 - 송신 측(클라이언트)이 수신 측(서버)으로부터 ack 이벤트 수신하여 처리
 - 이벤트 필드 내용 출력 외에 별도 태스크는 없음
 - 수정 사항 없음

수신 측(서버) GENERATOR 스레드 시작

- CMFileSyncManager
public Boolean isCompleteFileSync(String userName)
->
public Boolean isCompleteFileSync(String initiatorName, UUID initiatorUuid)
 - 메소드 역할
 - 해당 initiator 가 시작한 파일 동기화가 완료되었는지 여부를 판단
 - 수정 방안
 - 변수명 수정
newClientPathEntryList -> newInitiatorPathEntryList
m_cmInfo.getFileSyncInfo() -> syncInfo // 앞에서 syncInfo 설정
 - 수정 구현

- // get CMFileSyncGenerator object 아래 라인을 다음으로 수정
 CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
 CMFileSyncSenderKey senderKey = new CMFileSyncSenderKey(initiatorName, initiatorUuid);
 CMFileSyncGenerator syncGenerator = syncInfo.getSyncGeneratorMap().get(senderKey);
 ...
- // get basis file list 아래 두 라인을 다음으로 수정
 UUID initiatorDeviceUuid = syncGenerator.getInitiatorDeviceUuid();
 CMFileSyncStateKey stateKey = new FileSyncStateKey(initiatorName, initiatorDeviceUuid);
 basisFileList = syncInfo.getBasisFileListMap().get(stateKey);
 ...
- // compare the number of files of which sync is completed ... 아래 라인 다음으로 수정
 fileEntryList = syncInfo.getInitiatorPathEntryListMap().get(stateKey);
 ...
- 나머지 기존 코드 동일
 ...
- CMFileSyncGenerator
 - 멤버 변수 수정
 private List<CMFileSyncEntry> newClientPathEntryList;
 ->
 private List<CMFileSyncEntry> **newInitiatorPathEntryList;**
 - 생성자 초기화, getter/setter 수정
 - 해당 변수명 사용하는 곳 수정
- Private List<Path> createBasisFileList()**
 - 메소드 역할
 - rsync 에서 수신 측(서버)이 유지하는 파일 리스트 생성
 - CMFileSyncManager 의 createPathList() 호출해서 생성
 - 수정 방안
 - 양방향 동기화 시에, 동기화 홈 디렉토리를 구하는 과정에서 현재 사용자가 서버인지 클라이언트인지에 따라 홈 디렉토리를 다르게 하도록 수정 필요
 - 수정 구현
 - // get the server sync home 전까지 기존 코드 동일
 ...
 - // get the server sync home 과 아래 라인을 다음으로 수정
 // get the sync home
 CMConfigurationInfo confInfo = CMConfigurationInfo.getInstance();
 Path syncHome;

```

if( confInfo.getSystemType().equals("SERVER") ) {
    syncHome = syncManager.getServerSyncHome(initiatorName);
}
else {
    syncHome = syncManager.getClientSyncHome();
}
...

```

- 나머지 serverSyncHome 을 syncHome 으로 수정

...

public void run()

■ 메소드 역할

- 송신 측(클라이언트)이 보낸 파일 목록과 자신이 가지고 있는 basis 파일 목록 비교
- 자신에게 없는 파일은 송신측에게 신규 파일 전송 요청
- 양쪽에 있는 파일 중 동기화 필요한 파일들의 블록 체크섬을 송신 측으로 전송

■ 수정 방안

- 멤버 변수 newClientPathEntryList 이름 변경 -> newInitiatorPathEntryList
- 멤버 변수 userName 대신 initiatorName 사용
- 신규 파일 요청 및 블록 체크섬 전송 관련 이벤트 필드의 변경 사항 적용
 - 이벤트 필드에 공통 initiatorName, initiatorUuid, initiatorDeviceUuid 추가
 - 이벤트 필드에 Sender, receiver, userName 설정은 삭제

■ 수정 구현

- 메소드에 사용된 멤버 변수 이름 수정
 newClientPathEntryList -> newInitiatorPathEntryList
 userName -> initiatorName
- basisFileListMap.put(...) 전까지 기존 코드 동일
 ...
- basisFileListMap.put(...) 라인을 다음으로 수정
 CMFileSyncStateKey key = new CMFileSyncStateKey(initiatorName, initiatorDeviceUuid);
 basisFileListMap.put(key, basisFileList);
- if(syncManager.isCompleteFileSync(...)) 전까지 기존 코드 동일
 ...
- if() 문 다음으로 수정
 if(syncManager.isCompleteFileSync(initiatorName, initiatorUuid)) {
 syncManager.completeFileSync(initiatorName, initiatorUuid); // 파일 업데이트 완료

과정에서 수정 구현 예정
}

REQUEST_NEW_FILES 송신

- CMFileSyncGenerator

Private boolean requestTransferOfNewFiles()

■ 메소드 역할

- Generator 객체 생성 후 run() 내에서 호출
- 수신 측(서버)에 필요한 신규 디렉토리를 먼저 생성하고, 나머지 신규 파일에 대해 단계적으로 전송 요청 이벤트를 송신 측(클라이언트)으로 전송
- 이 메소드 수신 이후 송신 측은 CM 파일 전송 프로토콜을 통해 요청 파일들 순차적으로 전송

■ 수정 방안

- syncHome 을 현재 서버/클라이언트 여부에 따라 설정 (양방향 동기화 대비)
- 멤버 변수 이름 수정
newClientPathEntryList -> newInitiatorPathEntryList
userName -> initiatorName
- syncManager.completeNewFileTransfer()에 initiatorUuid 추가 (initiator 가 서버이면 uuid 는 null 가능성 있음)
- CMFileSyncEventRequestNewFiles 이벤트 설정 수정
 - 공통 필드 추가: initiatorName, initiatorUuid, initiatorDeviceUuid
 - Sender, receiver 설정 삭제 (나중에 공통 설정 예정)
 - Requester name 설정 삭제: requester name 필드 자체를 이벤트에서 삭제
- unicastEvent()에 initiator uuid 파라미터 추가

■ 수정 구현

- // get sync home 전까지 기존 코드 동일
// 멤버 변수명만 수정
// newClientPathEntryList -> newInitiatorPathEntryList
...
● // get sync home 아래 동기화 홈 구하기 다음으로 수정
CMFileSyncManager syncManager = ...
Objects.requireNonNull(syncManager);
Path syncHome;

```

CMConfigurationInfo confInfo = CMConfigurationInfo.getInstance();
if( confInfo.getSystemType().equals("SERVER") ) {
    syncHome = syncManager.getServerHome( initiatorName );
}
else if( confInfo.getSystemType().equals("CLIENT") ) {
    syncHome = syncManager.getClientHome();
}
else {
    // 에러 메시지 출력 (wrong system type!)
}

```

- // create new sub-directories ... 해당 코드 다음으로 수정
 // 기존 코드에서 아래 사항만 수정
 // newClientPathEntryList -> newInitiatorPathEntryList
 // userName -> initiatorName
 ...
 ...completeNewFileTransfer(...) 라인을 다음으로 수정
 Boolean ret = syncManager.completeNewFileTransfer(initiatorName, initiatorUuid);
 ...
- While(numRequestedComplete < ...) 문을 다음으로 수정
 while(numRequestedCompleted < newFileEntryList().size()) {
 // create a request event
 CMFileSyncEventRequestNewFiles fse = new CMFileSyncEventRequestNewFiles();
 // 공통 필드 설정
 fse.setInitiatorName(initiatorName);
 fse.setInitiatorUuid(initiatorUuid);
 fse.setInitiatorDeviceUuid(initiatorDeviceUuid);
 //// set numRequestedFiles and requestedFileList
 (기존 코드 동일)
 ...
 // send the request event
 sendResult = CMEventManager.unicastEvent(fse, initiatorName, initiatorUuid);
 ...
 }
- Return true;

SKIP_UPDATE_FILE 송신

- CMFileSyncManager
public Boolean skipUpdateFile(String initiatorName, UUID initiatorUuid)

■ 메소드 역할

- 아래 compareBasisAndClientFileList() 메소드 내에서 호출
- 수신 측(서버)에서 특정 파일의 동기화(업데이트)를 수행하지 않기로 결정했을 때, 내부 상태를 “완료”로 갱신하고, 송신 측(클라이언트)에게 해당 파일은 동기화가 필요 없음을 알리는 이벤트를 전송

■ 수정 방안

- SKIP_UPDATE_FILE 이벤트 필드 수정
userName 필드 삭제, 공통 필드 설정 (initiatorName, initiatorUuid, initiatorDeviceUuid)
- syncGeneratorMap 의 value 구할 때 userName 이 아닌 (initiatorName, initiatorUuid) key 로 수정
- syncHome 을 CM system type 에 따라 설정하는 것으로 수정
- unicastEvent() 매개 변수 수정

■ 수정 구현

- 디버그 메시지 수정 출력
// userName 대신 initiatorName, initiatorUuid 출력
...
● // get CMFileSyncGenerator 아래 라인 다음으로 수정
CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
CMFileSyncSenderKey key = new CMFileSyncSenderKey(initiatorName, initiatorUuid);
CMFileSyncGenerator syncGenerator = syncInfo.getSyncGeneratorMap().get(key);
...
● // set the isUpdateFileCompletedMap element 와 // update numUpdateFilesCompleted 해당
구문 기존 코드 동일
...
● // create a SKIP_UPDATE_FILE event 아래 이벤트 설정 구문 다음으로 수정
CMFileSyncEventSkipUpdateFile fse = new CMFileSyncEventSkipUpdateFile();
// 공통 필드 설정
fse.setInitiatorName(initiatorName);
fse.setInitiatorUuid(initiatorUuid);
fse.setInitiatorDeviceUuid(syncGenerator.getInitiatorDeviceUuid());
// get the relative path of the basis file path
Path syncHome;
CMConfigurationInfo confInfo = CMConfigurationInfo.getInstance();
if(confInfo.getSystemType().equals("SERVER")) {
 syncHome = getServerSyncHome(initiatorName);
}
else {
 syncHome = getClientSyncHome();
}
Path relativePath = basisFile.subpath(syncHome.getNameCount(), basisFile.getNameCount());
// 나머지 필드 설정 (set the relative path to the event)
fse.setSkippedPath(relativePath);
● Return 구문을 다음으로 수정
return CMEEventManager.unicastEvent(fse, initiatorName, initiatorUuid);

START_FILE_BLOCK_CHECKSUM 송신

- CMFileSyncGenerator

Private Boolean compareBasisAndClientFileList() -> compareBasisAndInitiatorFileList()

■ 메소드 역할

- Generator 생성 후 run() 내에서 호출
- 수신 측(서버)과 송신 측(클라이언트)의 파일 상태를 비교하여, 이미 동기화된 파일은 건너뛰고, 필요한 파일만 블록 단위 체크섬을 계산하여 송신측에게 전달

■ 수정 방안

- 변수명 수정
userName -> initiatorName
serverSyncHome -> syncHome
clientPathEntry -> initiatorPathEntry
clientPathEntryIndex -> initiatorPathEntryIndex
clientPathEntryList -> initiatorPathEntryList
- 동기화 홈 구하기 수정
서버 기준 동기화 홈이 아닌, 현재 시스템 타입(서버 또는 클라이언트)에 따른 동기화 홈 구하기
- skipUpdateFile() 메소드 파라미터에 initiatorUuid 추가 수정

■ 수정 구현

- List<Path> basisFileList = ... 전까지 기존 코드 동일
...
- List<Path> basisFileList = ... 라인을 다음으로 수정
CMFileSyncStateKey stateKey = new CMFileSyncStateKey(initiatorName, initiatorDeviceUuid);
List<Path> basisFileList = syncInfo.getBasisFileListMap().get(stateKey);
...
- // get the server sync home 코멘트 및 아래 라인 다음으로 수정
// get the file sync home
CMConfigurationInfo confInfo = CMConfigurationInfo.getInstance();
Path syncHome;
if(confInfo.getSystemType().equals("SERVER")) {
 syncHome = syncManager.getServerSyncHome(initiatorName);
}
else {
 syncHome = syncManager.getClientSyncHome();
}
...
- List<CMFileSyncEntry> clientPathEntryList = ... 라인을 다음으로 수정
List<CMFileSyncEntry> initiatorPathEntryList =

```
syncHome.getInitiatorPathEntryListMap().get(stateKey);
...
```

- // already synchronized 아래 라인을 다음으로 수정

```
syncManager.skipUpdateFile(initiatorName, initiatorUuid, basisFile);
...
```
- // complete the update-file task of the file-sync for this sub-directory 아래 구문을 다음으로 수정

```
boolean ret = syncManager.completeUpdateFile(initiatorName, initiatorUuid, basisFile);
if(!ret) {
    // 에러 메시지에 initiatorName, initiatorUuid 로 수정
}
...
```
- 나머지 기존 코드 동일

```
...
```

Private Boolean sendBlockChecksum(int initiatorFileEntryIndex, CMFileSyncBlockChecksum[] checksumArray)

■ 메소드 역할

- 블록 체크섬 전송 시작을 알리는 START_FILE_BLOCK_CHECKSUM 이벤트를 송신 측(클라이언트)으로 전송

■ 수정 방안

- 이벤트 필드 수정
 - Sender, receiver 설정 삭제 (unicastEvent()에서 자동 설정)
 - 공통 필드 (initiatorName, initiatorUuid, initiatorDeviceUuid) 설정 추가
- unicastEvent() 파라미터 수정 (initiatorName, initiatorUuid 지정)

■ 수정 구현

- // create a START_FILE_BLOCK_CHECKSUM event 아래 이벤트 설정 다음으로 수정

```
CMFileSyncEventStartFileBlockChecksum fse = new CMFileSyncEventStartFileBlockChecksum();
// 공통 필드 설정 추가
fse.setInitiatorName(initiatorName);
fse.setInitiatorUuid(initiatorUuid);
fse.setInitiatorDeviceUuid(initiatorDeviceUuid);
// 나머지 필드 설정 기존 코드 동일
...
```
- // send the event 아래 라인 다음으로 수정

```
Boolean ret = CMEventManager.unicastEvent(fse, initiatorName, initiatorUuid);
...
```

REQUEST_NEW_FILES 수신

- CMFileSyncEventHandler

Private Boolean processREQUEST_NEW_FILES(CMFileSyncEvent fse)

■ 메소드 역할

- 수신 측(서버)에서 신규 파일 전송을 요청함
- 이를 처리하는 송신 측(클라이언트)은 요청 파일을 CM 파일 전송 프로토콜로 전송
- 파일 전송이 완료되면 수신 측은 파일 동기화 완료 여부 검사

■ 수정 방안

- clientSyncHome 대신 현재 CM system type(서버 여부)에 따라 동기화 홈 다르게 구하도록 수정 (양방향 동기화 대비)
- requesterName 은 이벤트 헤더의 sender 로 설정
- requesterUuid 변수를 이벤트 헤더의 senderUuid 로 추가 설정
- CMFileTransferManager.pushFile() 호출시 매개변수 수정 (requesterUuid 매개 변수 추가)

■ 수정 구현

- // get the client sync home 아래 라인을 다음으로 수정
...
CMConfigurationInfo confInfo = CMConfigurationInfo.getInstance();
Path syncHome;
if(confInfo.getSystemType().equals("SERVER")) {
 syncHome = syncManager.getServerSyncHome(fse_rnf.getSender()); // 확인 필요
}
else {
 syncHome = syncManager.getClientSyncHome();
}
}
- // get the requester name 아래 라인을 다음으로 수정
String requesterName = fse_rnf.getSender();
UUID requesterUuid = fse_rnf.getSenderUuid();
- For(Path path : requestedFileList) ... 까지 기존 코드 동일
...
- For 문 블록 내 코드를 다음으로 수정
for(Path path : requestedFileList) {
 Path syncPath = syncHome.resolve(path); // adjust the path with the sync home
 if(!CMFileTransferManager.pushFile(syncPath.toString(), requesterName, requesterUuid);
 sendResult = false;
 }
...
}

수신 측(서버) 파일 전송 완료 후 동기화용인지 확인

- CMFileSyncManager

Public void checkNewTransferForSync(CMFileEvent fe)

■ 메소드 역할

- 파일을 수신할 때마다 호출하며, 파일 동기화 수신 측(서버)이 동기화용 신규 파일 전송인지 여부를 체크하는 메소드
- 파일 동기화용 신규 파일 전송이라면, 하나의 신규 파일 전송 완료 처리를 하고 동기화가 완료된건지의 여부를 검사하여 동기화 완료 여부를 처리

■ 수정 방안

- fileSender 이외에 fileSenderUuid 구하기
- 변수/메소드명 수정
getClientPathEntryList() -> getNewInitiatorPathEntryList()
newClientPathEntryList -> newInitiatorPathEntryList
- serverSyncHome 대신 CM system type 에 따라 동기화 홈 syncHome 구하기
- 메소드에 fileSenderUuid 파라미터 추가
completeNewFileTransfer(), isCompleteFileSync(), completeFileSync()

■ 수정 구현

- // get the new file list 아래 구문을 다음으로 수정
String fileSender = fe.getFileSender();
UUID fileSenderUuid = fe.getFileSenderUuid();
CMFileSyncSenderKey senderKey = new CMFileSyncSenderKey(fileSender, fileSenderUuid);
CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
CMFileSyncGenerator syncGenerator = syncInfo.getSyncGeneratorMap().get(senderKey);
if(syncGenerator == null) {
 // 에러 메시지에 fileSenderUuid 추가
 ...
}
List<CMFileSyncEntry> newInitiatorPathEntryList =
syncGenerator.getNewInitiatorPathEntryList();
- // search for the entry in the newInitiatorPathEntryList 아래 구문 변수명만 수정
...
// newClientPathEntryList -> newInitiatorPathEntryList
...
- If(foundPath != null) 블록 내 // get the file-transfer home 아래 라인 다음으로 수정
CMConfigurationInfo confInfo = CMConfigurationInfo.getInstance();
Path transferFileHome = confInfo.getTransferredFileHome().resolve(fileSender);

- `// get the server sync home` 코멘트 및 아래 라인 다음으로 수정
`// get the sync home`
`Path syncHome;`
`if( conflInfo.getSystemType().equals("SERVER") ) {`
`syncHome = getServerSyncHome( fileSender );`
`}`
`else {`
`syncHome = getClientSyncHome();`
`}`
`...`
- `// complete the new-file-transfer` 아래 라인 다음으로 수정
`Boolean result = completeNewFileTransfer(fileSender, fileSenderUuid, foundPath);`
- `if(result)` 블록 다음으로 수정
`if(result) {`
`// remove the completed newInitiatorPathEntry from the list`
`... (newClientPathEntryList -> newInitiatorPathEntryList 변수명 수정)`
`// check if the file-sync is complete or not`
`if( isCompleteFileSync(fileSender, fileSenderUuid) ) {`
`// complete the file-sync task`
`completeFileSync(fileSender, fileSenderUuid);`
`}`
`}`
`...`

Public Boolean completeNewFileTransfer(String initiatorName, Uuid initiatorUuid, Path path)

■ 메소드 역할

- 수신 측(서버)이 하나의 신규 파일 전송이 완료되었음을 (path, true) 로 신규 파일 전송 완료 여부 관리 map 에 기록
- 수신 측(서버)이 송신 측(클라이언트)으로 하나의 신규 파일 전송이 완료되었음을 이벤트 전송으로 통지

■ 수정 방안

- syncGenerator 구하기 수정
 - CMFileSyncSenderKey (initiatorName, initiatorUuid)로 generator map 에서 값 구하기
- COMPLETE_NEW_FILE 이벤트 필드 설정 수정
 - Sender, receiver, userName 설정 삭제
 - 공통 필드 설정 (initiatorName, initiatorUuid, initiatorDeviceUuid)
 - unicastEvent() 파라미터 uuid 추가

■ 수정 구현

- `// get CMFileSyncGenerator 아래 라인을 다음으로 수정`
`CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();`
`CMFileSyncSenderKey senderKey = new CMFileSyncSenderKey(initiatorName, initiatorUuid);`
`CMFileSyncGenerator syncGenerator = syncInfo.getSyncGeneratorMap().get(senderKey);`
`...`
- `// create a COMPLETE_NEW_FILE event 아래 이벤트 필드 설정 부분 다음으로 수정`
`CMFileSyncEventCompleteNewFile fse = new CMFileSyncEventCompleteNewFile();`
`// 공통 필드 설정`
`fse.setInitiatorName( initiatorName );`
`fse.setInitiatorUuid( initiatorUuid );`
`fse.setInitiatorDeviceUuid( syncGenerator.getInitiatorDeviceUuid() );`
`// 나머지 필드 설정`
`... (기존 코드 동일)`
- `// send the event 아래 라인 다음으로 수정`
`Boolean ret = CMEventManager.unicastEvent(fse, initiatorName, initiatorUuid);`
`...`
- `Return true;`

Public boolean completeFileSync(String initiatorName, Uuid initiatorUuid)

■ 메소드 역할

- 동기화 수신 측(서버)이 송신 측(클라이언트)에게 파일 동기화가 완료되었음을 통보하고, 동기화 관련 내부 자료를 정리
- 내부에서 `sendCompleteFileSync()`, `deleteFileSyncInfo()` 메소드 호출

■ 수정 방안

- `sendCompleteFileSync()`, `deleteFileSyncInfo()` 호출 시 매개변수로 `initiatorUuid` 추가

■ 수정 구현

- 디버그 메시지에 `initiatorName`, `initiatorUuid` 포함
`...`
- `sendCompleteFileSync(...)` 호출 라인을 다음으로 수정
`result = sendCompleteFileSync(initiatorName, initiatorUuid);`
`...`
- `deleteFileSyncInfo(...)` 호출 라인을 다음으로 수정
`deleteFileSyncInfo(initiatorName, initiatorUuid);`
`...`

Private Boolean sendCompleteFileSync(String initiatorName, UUID initiatorUuid)

■ 메소드 역할

- 수신 측(서버)에서 송신 측(클라이언트)으로 파일 동기화가 완료되었다는 이벤트 전송
- 수정 방안
 - syncGenerator 구할 때 CMFileSyncSenderKey(initiatorName, initiatorUuid) 이용
 - CMFileSyncEventCompleteFileSync 이벤트 필드 설정 수정
 - 공통 필드 추가 (initiatorName, initiatorUuid, initiatorDeviceUuid)
 - unicastEvent()의 타겟 파라미터를 initiatorName, initiatorUuid 로 수정
- 수정 구현
 - 디버그 메시지에 initiatorName, initiatorUuid 포함
...
 - // get the CMFileSyncGenerator reference 아래 라인 다음으로 수정
 CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
 CMFileSyncSenderKey senderKey = new CMFileSyncSenderKey(initiatorName, initiatorUuid);
 CMFileSyncGenerator syncGenerator = syncInfo.getSyncGeneratorMap().get(senderKey);
 ...
 - // create a COMPLETE_FILE_SYNC event 아래 구문 다음으로 수정
 int numFilesCompleted = ... // 기존 코드 동일
 CMFileSyncEventCompleteFileSync fse = new CMFileSyncEventCompleteFileSync();
 // 공통 필드 설정
 fse.setInitiatorName(initiatorName);
 fse.setInitiatorUuid(initiatorUuid);
 fse.setInitiatorDeviceUuid(syncGenerator.getInitiatorDeviceUuid());
 // 나머지 필드 설정
 fse.setNumFilesCompleted(numFilesCompleted);
 - // send the event 아래 라인 다음으로 수정
 return CEventManager.unicastEvent(fse, initiatorName, initiatorUuid);

Private void deleteFileSyncInfo(String initiatorName, UUID initiatorUuid)

- 메소드 역할
 - 수신 측(서버)에서 파일 동기화가 완료된 initiator 의 정보를 CMFileSyncInfo 에서 삭제
- 수정 방안
 - CMFileSyncInfo 구하기 수정
 - initiatorPathEntryListMap, syncGeneratorMap 에서 삭제할 때 key 를 각각 CMFileSyncStateKey, CMFileSyncSenderKey 로 수정
- 수정 구현

- 디버그 메시지에 `initiatorName`, `initiatorUuid` 포함
...
- `// get CMFileSyncInfo reference` 아래 라인 다음으로 수정
`CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();`
- `// remove elements in fileEntryListMap` 아래 라인 다음으로 수정
`CMFileSyncSenderKey senderKey = new CMFileSyncSenderKey(initiatorName, initiatorUuid);`
`CMFileSyncGenerator syncGenerator = syncInfo.getSyncGeneratorMap().get(senderKey);`
`CMFileSyncStateKey stateKey = new CMFileSyncStateKey(initiatorName,`
`syncGenerator.getInitiatorDeviceUuid());`

`syncInfo.getInitiatorPathEntryListMap().remove(stateKey);`
- `// remove elements in syncGeneratorMap` 아래 라인 다음으로 수정
`syncInfo.getSyncGeneratorMap().remove(senderKey);`

COMPLETE_NEW_FILE 수신

- `CMFileSyncEventHandler`

Private Boolean processCOMPLETE_NEW_FILE(CMFileSyncEvent fse)

- 메소드 역할
 - 송신 측(클라이언트)이 수신 측(서버)으로부터 신규 파일 전송의 수신을 완료했다는 이벤트를 받아, 해당 파일의 동기화가 완료된 것으로 표시
- 수정 방안
 - `CMFileSyncInfo` 구하기
- 수정 구현
 - `CMFileSyncInfo syncInfo = ...` 라인을 다음으로 수정
`CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();`
...
 - `Return true;`

SKIP_UPDATE_FILE 수신

- `CMFileSyncEventHandler`

private Boolean processSKIP_UPDATE_FILE(CMFileSyncEvent fse)

- 메소드 역할
 - 수신 측(서버)이 보낸 업데이트 `skip` 할 파일 정보를 송신 측(클라이언트)에서 처리
 - 송신 측(클라이언트)에서의 업데이트 완료 파일 정보 갱신
- 수정 방안

- CMFileSyncInfo 구하기만 수정
- 수정 구현
 - // update info for the file-update completion at the client 아래 라인 다음으로 수정
CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
...

START_FILE_BLOCK_CHECKSUM 수신 및 START_FILE_BLOCK_CHECKSUM_ACK 송신

- CMFileSyncEventHandler

Private Boolean processSTART_FILE_BLOCK_CHECKSUM(CMFileSyncEvent fse)

- 메소드 역할
 - 해당 파일을 위해 블록 체크섬 배열과 “해시→블록 인덱스” 맵을 새로 만들어 CMFileSyncInfo 내부 자료구조에 등록
 - 수신 이벤트의 정보(블록 크기·파일 인덱스·총 블록 수)와 returnCode 를 담은 ACK 이벤트를 구성한 뒤 원 발신자에게 전송
- 수정 방안
 - CMFileSyncInfo 구하기 수정
 - CMFileSyncEventStartFileBlockChecksumAck 이벤트 필드 설정 수정
 - 공통 필드 추가 (initiatorName, initiatorUuid, initiatorDeviceUuid)
 - unicastEvent() 타겟 파라미터 수정 (sender uuid 추가)
- 수정 구현
 - // get the file in the client file entry list 및 아래 라인을 다음으로 수정
// get the file in the initiator file entry list
CMFileSyncInfo fsInfo = CMFileSyncInfo.getInstance();
...
 - // create an ack event 아래 이벤트 설정 구문을 다음으로 수정
CMFileSyncEventStartFileBlockChecksumAck ackEvent = new
CMFileSyncEventStartFileBlockChecksumAck();
// 공통 필드 설정
ackEvent.setInitiatorName(startChecksumEvent.getInitiatorName());
ackEvent.setInitiatorUuid(startChecksumEvent.getInitiatorUuid());
ackEvent.setInitiatorDeviceUuid(startChecksumEvent.getInitiatorDeviceUuid());
// set fields as they are in the received event
... (기존 코드 동일)

```
// set return code
... (기존 코드 동일)
```

- // send the ack event 아래 라인을 다음으로 수정
return CMEEventManager.unicastEvent(ackEvent, startChecksumEvent.getSender(),
startChecksumEvent.getSenderUuid());

START_FILE_BLOCK_CHECKSUM_ACK 수신 및 FILE_BLOCK_CHECKSUM / END_FILE_BLOCK_CHECKSUM 송신

- CMFileSyncEventHandler

Private Boolean processSTART_FILE_BLOCK_CHECKSUM_ACK(CMFileSyncEvent fse)

■ 메소드 역할

- 수신 측(서버)에서 FILE_BLOCK_CHECKSUM 이벤트를 송신 측(클라이언트)으로 필요한 경우 반복 전송
- 모든 블록 체크섬을 전송한 후, 마지막으로 END_FILE_BLOCK_CHECKSUM 전송

■ 수정 방안

- 변수명 변경
 - userName -> initiatorName
- CMFileSyncGenerator 구하기 수정
- CMFileSyncEventFileBlockChecksum 이벤트 필드 수정
 - 공통 필드 추가 (initiatorName, initiatorUuid, initiatorDeviceUuid)
 - userName 필드 삭제
- CMFileSyncEventEndFileBlockChecksum 이벤트 필드 수정
 - 공통 필드 추가 (initiatorName, initiatorUuid, initiatorDeviceUuid)
- unicastEvent() 타겟 파라미터 수정 (sender, senderUuid)

■ 수정 구현

- // get CMFileSyncGenerator reference 아래 구문을 다음으로 수정
String initiatorName = startAckEvent.getInitiatorName();
UUID initiatorUuid = startAckEvent.getInitiatorUuid();
CMFileSyncSenderKey senderKey = new CMFileSyncSenderKey(initiatorName, initiatorUuid);
CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
CMFileSyncGenerator syncGenerator = syncInfo.getSyncGeneratorMap().get(senderKey);
...

- `// create FILE_BLOCK_CHECKSUM event` 아래 이벤트 필드 설정 다음으로 수정
`CMFileSyncEventFileBlockChecksum checksumEvent = new CMFileSyncEventBlockChecksum();`
`// 공통 필드 설정`
`checksumEvent.setInitiatorName( initiatorName );`
`checksumEvent.setInitiatorUuid( initiatorUuid );`
`checksumEvent.setInitiatorDeviceUuid( startAckEvent.getInitiatorDeviceUuid() );`
`// 나머지 필드 설정`
`... (기존 코드 동일)`
- `// send the event` 아래 라인 다음으로 수정
`ret = CMEventManager.unicastEvent(checksumEvent, initiatorName, initiatorUuid);`
`...`
- `// create and send END_FILE_BLOCK_CHECKSUM event` 아래 이벤트 필드 설정 및 전송 구문
다음으로 수정
`CMFileSyncEventEndFileBlockChecksum endChecksumEvent = new`
`CMFileSyncEventEndFileBlockChecksum();`
`// 공통 필드 설정`
`endChecksumEvent.setInitiatorName( initiatorName );`
`endChecksumEvent.setInitiatorUuid( initiatorUuid );`
`endChecksumEvent.setInitiatorDeviceUuid( syncGenerator.getInitiatorDeviceUuid() );`
`// 나머지 필드 설정`
`... (기존 코드 동일)`
`ret = CMEventManager.unicastEvent(endChecksumEvent, initiatorName, initiatorUuid);`
`...`
- `Return true;`

FILE_BLOCK_CHECKSUM 수신

- `CMFileSyncEventHandler`

Private Boolean processFILE_BLOCK_CHECKSUM(CMFileSyncEvent fse)

■ 메소드 역할

- 수신 측(서버)이 보낸 특정 파일 엔트리의 블록 체크섬 배열의 일부 또는 전체를, 송신 측(클라이언트)이 로컬 블록 체크섬 배열로 복사

■ 수정 방안

- 수정 사항 없음

END_FILE_BLOCK_CHECKSUM 수신 및 UPDATE_EXISTING_FILE, END_FILE_BLOCK_CHECKSUM_ACK 송신

- `CMFileSyncEventHandler`

Private Boolean processEND_FILE_BLOCK_CHECKSUM(CMFileSyncEvent fse)

■ 메소드 역할

- 송신 측(클라이언트)이 수신 측(서버)으로부터 한 파일 엔트리의 모든 블록 체크섬 수신을 완료
- 송신 측 로컬의 해당 파일을 블록 단위로 비교하여, 동일한 블록은 블록 번호를, 매칭 블록이 없는 경우는 블록 내용을 순차적으로 수신 측으로 전송
- 파일 비교 작업이 끝나면 `END_FILE_BLOCK_CHECKSUM_ACK` 이벤트를 수신 측으로 전송

■ 수정 방안

- `Sender, receiver` 로컬 변수 삭제
- `compareFileBlocks()` 메소드 수정
 - 내부에서 `UPDATE_EXISTING_FILE` 이벤트 전송
- `CMFileSyncEventEndFileBlockChecksumAck` 이벤트 필드 수정
 - `Sender, receiver` 삭제
 - 공통 필드 (`initiatorName, initiatorUuid, initiatorDeviceUuid`) 설정 추가

■ 수정 구현

- `String sender, String receiver` 로컬 변수 삭제
...
- `// get the local path list` 아래 라인을 다음으로 수정
`CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();`
`List<Path> pathList = syncInfo.getPathList();`
- `// create and send an END_FILE_BLOCK_CHECKSUM_ACK event` 아래 이벤트 설정 및 전송
구문 다음으로 수정
`CMFileSyncEventEndFileBlockChecksumAck ackEvent = new`
`CMFileSyncEventEndFileBlockChecksumAck();`
`// 공통 필드 설정`
`ackEvent.setInitiatorName( endChecksumEvent.getInitiatorName() );`
`ackEvent.setInitiatorUuid( endChecksumEvent.getInitiatorUuid() );`
`ackEvent.setInitiatorDeviceUuid( endChecksumEvent.getInitiatorDeviceUuid() );`
`// 나머지 필드 설정`
`...(기존 코드 동일)`
`// send the ack event`
`ret = CMEventManager.unicastEvent( ackEvent, endChecksumEvent.getSender(),`
`endChecksumEvent.getSenderUuid() );`
...
- `Return true;`

Private Boolean compareFileBlocks(CMFileSyncEventEndFileBlockChecksum endChecksumEvent)

■ 메소드 역할

- 송신 측(클라이언트)은 수신한 base 파일의 블록 체크섬과 로컬의 파일을 비교하여, 일치 블록과 불일치 블록을 검색
- 일치 블록이 나오거나, 불일치 블록 버퍼가 가득차면 UPDATE_EXISTING_FILE 이벤트를 수신 측(서버)으로 전송
- 비교가 끝나고 마지막 남은 바이트들에게 대해서 불일치 블록으로 동일 이벤트 전송

■ 수정 방안

- CMFileSyncInfo 구하기 수정
- sendUpdateExistingFileEvent(...) 메소드 수정

■ 수정 구현

- String receiver 변수 선언 아래 다음 변수 추가
...
UUID receiverUud = endChecksumEvent.getSenderUuid();
String initiatorName = endChecksumEvent.getInitiatorName();
UUID initiatorUuid = endChecksumEvent.getInitiatorUuid();
UUID initiatorDeviceUuid = endChecksumEvent.getInitiatorDeviceUuid();
- // get the local path list 아래 라인 다음으로 수정
CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
List<Path> pathList = syncInfo.getPathList();
- If(matchBlockIndex >= 0) 문 블록 내 sendUpdateExistingFileEvent() 메소드 다음으로 수정
...
Boolean ret = sendUpdateExistingFileEvent(receiver, receiverUuid, initiatorName,
initiatorUuid, initiatorDeviceUuid, fileEntryIndex, nonMatchBuffer, matchBlockIndex);
...
- 다음 else 블록 내 sendUpdateExistingFileEvent() 메소드 다음으로 수정
...
if(!nonMatchBuffer.hasRemaining()) {
 Boolean ret = sendUpdateExistingFileEvent(receiver, receiverUuid, initiatorName,
 initiatorUuid, initiatorDeviceUuid, fileEntryIndex, nonMatchBuffer, -1);
 ...
}
...
- 마지막 블록이 매칭되지 않았을 때의 처리인 if(!bBlockMatch) 블록 내
sendUpdateExistingFileEvent() 메소드 호출 라인을 다음으로 수정
if(!bBlockMatch) {

```

...
while( buffer.hasRemaining() ) {
    if(buffer.remaining() > nonMatchBuffer.remaining() ) {
        ...
        Boolean ret = sendUpdateExistingFileEvent(receiver, receiverUuid, initiatorName,
initiatorUuid, initiatorDeviceUuid, fileEntryIndex, nonMatchBuffer, -1);
        ...
    }
    else {
        ...
        Boolean ret = sendUpdateExistingFileEvent(receiver, receiverUuid, initiatorName,
initiatorUuid, initiatorDeviceUuid, fileEntryIndex, nonMatchBuffer, -1);
        ...
    }
}
}

```

Private Boolean sendUpdateExistingFileEvent(String sender, String receiver, int fileEntryIndex, ByteBuffer nonMatchBuffer, int matchBlockIndex)

->

private Boolean sendUpdateExistingFileEvent(String receiver, UUID receiverUuid, String initiatorName, UUID initiatorUuid, UUID initiatorDeviceUuid, int fileEntryIndex, ByteBuffer nonMatchBuffer, int matchBlockIndex)

■ 메소드 역할

- 송신 측(클라이언트)이 수신 측(서버)으로 업데이트된 파일의 delta 정보와 매칭 블록 정보를 담은 UPDATE_EXISTING_FILE 이벤트 전송

■ 수정 방안

- UPDATE_EXISTING_FILE 이벤트 필드 수정
 - 공통 필드 (initiatorName, initiatorUuid, initiatorDeviceUuid) 추가
 - Sender, receiver 설정은 삭제 (unicastEvent()에서 자동 설정)

■ 수정 구현

- /////// create and send an UPDATE_EXISTING_FILE event to the server 포함 아래 이벤트 필드 설정 부분 다음으로 수정
 /////// create and send an UPDATE_EXISTING_FILE event to the sync receiver
 CMFileSyncEventUpdateExistingFile updateEvent = new CMFileSyncEventUpdateExistingFile();
 // 공통 필드 설정
 updateEvent.setInitiatorName(initiatorName);
 updateEvent.setInitiatorUuid(initiatorUuid);
 updateEvent.setInitiatorDeviceUuid(initiatorDeviceUuid);
 // 나머지 필드 설정
 ... (기존 코드 동일)

- `// send the event` 아래 라인 다음으로 수정
`Boolean ret = CMEventManager.unicastEvent(updateEvent, receiver, receiverUuid);`
`...`
- `Return true;`

UPDATE_EXISTING_FILE 수신

- CMFileSyncEventHandler

Private Boolean processUPDATE_EXISTING_FILE(CMFileSyncEvent fse)

■ 메소드 역할

- 송신 측(클라이언트)이 특정 파일의 업데이트 내용을 이벤트로 전송한 것을 수신 측(서버)이 받아서 해당 파일의 내용을 임시 파일로 쓰며 업데이트
- 업데이트는 `nonMatchBuffer` 의 내용을 임시 파일로 쓰거나 `matchingBlock` 번호에 해당하는 블록을 `basis` 파일에서 찾아서 해당 블록을 임시 파일로 쓰는 것

■ 수정 방안

- CMFileSyncGenerator 구하기 수정
 - `(initiatorName, initiatorUuid)` pair 인 CMFileSyncSenderKey 를 key 로 사용
- basisFileListMap 구하기 수정
 - `(initiatorName, initiatorDeviceUuid)` pair 인 CMFileSyncStateKey 를 key 로 사용

■ 수정 구현

- 변수 선언 추가
`...`
`String initiatorName = updateEvent.getInitiatorName();`
`UUID initiatorUuid = updateEvent.getInitiatorUuid();`
`UUID initiatorDeviceUuid = updateEvent.getInitiatorDeviceUuid();`
- `// get the server sync home` 및 아래 구문을 다음으로 수정
`// get the sync manager`
`CMInfo cmlInfo = CMInfo.getInstance();`
`CMFileSyncManager syncManager = cmlInfo.getServiceManager(CMFileSyncManager.class);`
- `// get the basis file path` 아래 라인 generator 구하기를 다음으로 수정
`CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();`
`CMFileSyncSenderKey senderKey = new CMFileSyncSenderKey( initiatorName, initiatorUuid );`
`CMFileSyncGenerator syncGenerator = syncInfo.getSyncGeneratorMap().get(senderKey);`
`...`
- `List<Path> basisFileList = ...` 라인을 다음으로 수정
`CMFileSyncStateKey stateKey = new CMFileSyncStateKey( initiatorName, initiatorDeviceUuid );`


```
List<Path> basisFileList = syncInfo.getBasisFileListMap().get(stateKey);
...
```

- Return true;

END_FILE_BLOCK_CHECKSUM_ACK 수신

- CMFileSyncEventHandler

Private Boolean processEND_FILE_BLOCK_CHECKSUM_ACK(CMFileSyncEvent fse)

■ 메소드 역할

- 수신 측(서버)이 END_FILE_BLOCK_CHECKSUM_ACK 이벤트를 받으면 반환 코드 검증, 파일 채널 닫기 및 삭제, 파일 속성 갱신과 체크섬 검증 후 임시 파일을 기존 파일로 덮어쓰거나 수정 시간만 맞추고, 최종적으로 동기화 작업 완료 처리를 수행

■ 수정 방안

- M_cmInfo 대싱 CMInfo 싱글톤 인스턴스 구하기
- syncGenerator 구하기 수정
 - CMFileSyncSenderKey (initiatorName, initiatorUuid) 이용
- basisFileList 구하기 수정
 - CMFileSyncStateKey (initiatorName, initiatorDeviceUuid) 이용
- clientFileEntry 변수명 및 구하기 수정
 - initiatorFileEntry 로 수정
 - initiatorPathEntryListMap 의 value 는 CMFileSyncStateKey 를 key 로 이용
- completeUpdateFile(...) 기존 수정 부분에 이벤트 설정 추가 수정
- completeFileSync() 그 안의 sendCompleteFileSync() 수정

■ 수정 구현

- CMInfo, CMFileSyncInfo 변수 추가


```
CMInfo cmInfo = CMInfo.getInstance();
CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
```
- // get the sync generator reference 아래 구문 다음으로 수정


```
// initiator 정보 변수 선언
String initiatorName = ackEvent.getInitiatorName();
UUID initiatorUuid = ackEvent.getInitiatorUuid();
UUID initiatorDeviceUuid = ackEvent.getInitiatorDeviceUuid();
// get generator
```

```
CMFileSyncSenderKey senderKey = new CMFileSyncSenderKey(initiatorName, initiatorUuid);
CMFileSyncGenerator syncGenerator = syncInfo.getSyncGeneratorMap().get(senderKey);
```

- List<Path> basisFileList = ... 라인을 다음으로 수정

```
...
CMFileSyncStateKey stateKey = new CMFileSyncStateKey(initiatorName, initiatorDeviceUuid);
List<Path> basisFileList = syncInfo.getBasisFileListMap().get(stateKey);
...
```

- // get the client file entry reference 및 아래 라인을 다음으로 수정
// get the initiator file entry reference

```
CMFileSyncEntry initiatorFileEntry = Optional.of(syncInfo)
    .map(CMFileSyncInfo::getInitiatorPathEntryListMap)
    .map(map -> map.get(stateKey))
    .map(list -> list.get(fileEntryIndex))
    .orElse(null);
// 이후 코드도 clientFileEntry -> initiatorFileEntry 로 변경
...
```

- // complete the update-existing-file task 아래 구문 다음으로 수정
Boolean result = syncManager.completeUpdateFile(initiatorName, initiatorUuid, basisFilePath);
if(result) {
 // check if the file-sync is complete or not
 if(syncManager.isCompleteFileSync(initiatorName, initiatorUuid)) {
 // complete the file-sync task
 syncManager.completeFileSync(initiatorName, initiatorUuid);
 }
}
}
- Return true;

- CMFileSyncManager

Public boolean completeUpdateFile(String initiatorName, UUID initiatorUuid, Path path)

■ 메소드 역할

- 수신 측(서버)이 한 파일의 업데이트 동기화가 완료되었음을 (path, true)로 map 에 기록
- 송신 측(클라이언트)에게 한 파일의 업데이트 동기화 완료 사실을 COMPLETE_UPDATE_FILE 이벤트 전송으로 통지

■ 수정 방안

- CMFileSyncGenerator 구하기 수정
- COMPLETE_UPDATE_FILE 이벤트 필드 설정 수정
 - Sender, receiver, user name 설정 삭제
 - 공통 필드 설정 추가 (initiatorName, initiatorUuid, initiatorDeviceUuid)

- Sync home 구하기 수정
 - 서버의 동기화 디렉토리가 아닌, 추후 양방향 동기화시에 클라이언트도 될 수 있기 때문에 현재 CM system type 에 따라 동기화 홈 디렉토리 구하는 것으로 수정
- unicastEvent()에 uuid 파라미터 추가

■ 수정 구현

- // get CMFileSyncGenerator 아래 라인 다음으로 수정
 CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
 CMFileSyncSenderKey senderKey = new CMFileSyncSenderKey(initiatorName, initiatorUuid);
 CMFileSyncGenerator syncGenerator = syncInfo.getSyncGeneratorMap().get(senderKey);
 ...
- // create a COMPLETE_UPDATE_FILE event 아래 이벤트 필드 설정 다음으로 수정
 CMFileSyncEventCompleteUpdateFile fse = new CMFileSyncEventCompleteUpdateFile();
 // sender, receiver, userName 설정 삭제
 ...
 // 공통 필드 설정
 fse.setInitiatorName(initiatorName);
 fse.setInitiatorUuid(initiatorUuid);
 fse.setInitiatorDeviceUuid(syncGenerator.getInitiatorDeviceUuid());
 // 나머지 필드 설정
 ... (기존 코드 동일)
- // get the relative path of the basis file path 아래 동기화 홈 구하기 라인 다음으로 수정
 CMConfigurationInfo confInfo = CMConfigurationInfo.getInstance();
 Path syncHome;
 if(confInfo.getSystemType().equals("SERVER")) {
 syncHome = getServerSyncHome(initiatorName);
 }
 else {
 syncHome = getClientSyncHome();
 }
 ...
- 마지막 unicastEvent() 호출 라인 다음으로 수정
 return CMEventManager.unicastEvent(fse, initiatorName, initiatorUuid);

Public Boolean completeFileSync(String initiatorName, UUID initiatorUuid)

■ 메소드 역할

- 수신 측(서버) 파일 전송 완료 후 동기화 완료 처리에서 이미 수정 설계

COMPLETE_UPDATE_FILE 수신

- CMFileSyncEventHandler

Private Boolean processCOMPLETE_UPDATE_FILE(CMFileSyncEvent fse)

- 메소드 역할
 - 송신 측(클라이언트)이 수신 측(서버)으로부터 한 파일의 업데이트 동기화가 완료되었음을 통지받아, 자신의 로컬 정보 (complete map) 갱신
- 수정 방안
 - CMFileSyncInfo 구하기 수정
- 수정 구현
 - // update info for the file-update completion at the client 및 아래 라인 다음으로 수정
 // update info for the file-update completion at the sync sender
 CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
 ...
 - Return true;

COMPLETE_FILE_SYNC 수신

- CMFileSyncEventHandler
 - Private Boolean processCOMPLETE_FILE_SYNC(CMFileSyncEvent fse)**
 - 메소드 역할
 - 송신 측(클라이언트)이 수신 측(서버)으로부터 파일 동기화 세션 완료되었음을 통지받아, 자신의 로컬 정보 검사, 업데이트 및 초기화
 - 현 동기화 세션 사이에 watch service 통해 파일 변경이 탐지 되었는지 여부에 따라 새로운 동기화 진행 여부 판단
 - 수정 방안
 - CMFileSyncInfo 구하기 수정
 - 수정 구현
 - // compare event field (number of completed files) to the size of local sync-completion Map
 아래 라인을 다음으로 수정
 CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
 ...

온라인 모드로 변경 과정에서의 이벤트 송수신 처리 수정

- 프로토콜
 - 시작 시점
 - CMFileSyncManager 의 requestOnlineMode() 메소드 호출

- 이 메소드는 클라이언트 app 에서 바로 호출하거나 파일 동기화 proactive 모드에서 디렉토리 활성화 정도에 따라 자동 호출

■ 사용 이벤트

- ONLINE_MODE_LIST (client -> server)
- ONLINE_MODE_LIST_ACK (server -> client)
- END_ONLINE_MODE_LIST (client -> server)
- END_ONLINE_MODE_LIST_ACK (server -> client)

■ 수정 방안

- 사용 이벤트들에 공통인 requester 필드는 삭제하고 initiator 공통 필드 (initiatorName, initiatorUuid, initiatorDeviceUuid) 추가
- 현재 파일 모드 변경의 initiator 는 클라이언트
- 다중 로그인 상황에서 클라이언트들의 모드 변경 프로토콜은 별도 문서에서 설계 예정

ONLINE_MODE_LIST 송신

- CMFileSyncManager

private List<Path> toRelativePathList(List<Path> paths, Path basePath)

■ 메소드 역할

- 경로의 리스트를 basePath 기준 상대 경로로 변경
- 파라미터가 이미 상대경로이면 그대로 둠

■ 구현 내용

```
private List<Path> toRelativePathList(List<Path> paths, Path basePath) {
    Objects.requireNonNull(paths);
    Objects.requireNonNull(basePath);
    Path normalizedBasePath = basePath.toAbsolutePath().normalize();
    List<Path> relativePaths = new ArrayList<>(paths.size());

    for (Path path : paths) {
        Objects.requireNonNull(path);
        Path normalizedPath = path.normalize();
        Path relativePath;
        if (path.isAbsolute()) {
            Path normalizedAbsolutePath = normalizedPath.toAbsolutePath().normalize();
            if (!normalizedAbsolutePath.startsWith(normalizedBasePath)) {
                throw new IllegalArgumentException(
                    String.format("Path %s is not a descendant of base path %s", normalizedAbsolutePath,
normalizedBasePath));
            }
            relativePath = normalizedBasePath.relativeTo(normalizedAbsolutePath);
        } else {
            relativePath = normalizedPath;
        }
        relativePaths.add(relativePath);
    }
}
```

```

    return relativePaths;
}

```

- CMFileSyncManager

Private List<Path> createSubPathListForEvent(int initialByteNum, List<Path> relativePathList, int listIndex)

- 메소드 역할

- 파일들의 온라인/로컬 모드 변경 요청, 삭제 완료 파일들 정보 통지 등 path list 를 이벤트로 전송할 때, CM event 최대 크기 내에 담을 수 있는 서브 리스트를 구하는 메소드
 - 주어진 relativePathList 의 listIndex 매개 변수 index 의 path 원소부터 차례대로 처리

- 구현 방안

- 기존 createSubOnlineModeListForEvent(), createSubLocalModeListForEvent() 메소드 구현을 수정
 - 위 기존의 두 메소드를 호출하는 온라인/로컬 모드 요청 부분은 새로 구현한 이 메소드 호출로 변경

- 구현 내용

```

if(CMInfo._CM_DEBUG) {
    // 메소드명, initialByteNum, relativePathList, listIndex 출력
}

int curByteNum = initialByteNum;
List<Path> subList = new ArrayList<>();

Boolean ret = false;
for( int l = listIndex; l < relativePathList.size(); l++ ) {
    Path relativePath = relativePathList.get(l);
    curByteNum += CMInfo.STRING_LEN_BYTES_LEN + relativePath.toString().getBytes().length;
    if( curByteNum < CMInfo.MAX_EVENT_SIZE ) {
        ret = subList.add(relativePath);
        if( !ret ) {
            System.err.println("error to add " + relativePath);
            return null;
        }
    } else {
        break;
    }
}
return subList;

```

- CMFileSyncManager

Public Boolean requestOnlineMode(List<Path> pathList)

- 메소드 역할

- 클라이언트는 요청 파일 리스트를 상대 경로로 변경하여, 서버에게 이벤트로 통지
- 리스트의 경로 수가 많을 수 있으므로 이벤트가 담을 수 있는 최대 개수만큼만 파일 경로를 담아 보내고, 전송한 리스트는 온라인 모드 대기 큐로 이동
- 매개변수 리스트의 모든 경로 전송할 때까지 반복

■ 수정 방안

- CMFileSyncInfo 구하기 수정
- 변수명 userName 을 initiatorName 으로 수정
- 변수 initiatorUuid, initiatorDeviceUuid 추가
- 이벤트로 보낼 상대 경로 리스트 구하기 메소드 변경
 - createSubOnlineModeListForEvent() -> createSubPathListForEvent() 아래 수정 구현 참조
- ONLINE_MODE_LIST 이벤트 필드 설정 수정
 - Sender, receiver, requester 설정 삭제
 - 공통 필드 설정 추가 (initiatorName, initiatorUuid, initiatorDeviceUuid)
- unicastEvent() 파라미터 server uuid (null) 추가

■ 수정 구현

- CMFileSyncInfo syncInfo = ... 라인 다음으로 수정
 CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
 ...
- // get the user and server names 아래 관련 구문 다음으로 수정
 CMInteractionInfo interInfo = CMInteractionInfo.getInstance();
 String initiatorName = interInfo.getMyself().getName();
 UUID initiatorUuid = interInfo.getMyself().getUuid();
 UUID initiatorDeviceUuid = syncInfo.getDeviceUuid();
 String serverName = interInfo.getDefaultServerInfo().getServerName();
 ...
- While(listIndex < fileOnlyList.size()) 블록 다음으로 수정
 List<Path> relativeFileOnlyList = toRelativePathList(fileOnlyList, getClientSyncHome());

 while(listIndex < relativeFileOnlyList.size()) {
 // create an event
 CMFileSyncEventOnlineModeList listEvent = new CMFileSyncEventOnlineModeList();
 // 공통 필드 설정
 listEvent.setInitiatorName(initiatorName);
 listEvent.setInitiatorUuid(initiatorUuid);

```

listEvent.setInitiatorDeviceUuid( initiatorDeviceUuid );

// get relative path list to be added to this event 아래 라인을 다음으로 수정
List<Path> subList = createSubPathListForEvent( listEvent.getBytesNum(), relativeFileOnlyList,
listIndex);
...

// send the event
sendResult = CMEventManager.unicastEvent(listEvent, serverName, null);
...
}
...

```

- Return true;

ONLINE_MODE_LIST 수신 및 ONLINE_MODE_LIST_ACK 송신

- CMFileSyncEventHandler

Private Boolean processONLINE_MODE_LIST(CMFileSyncEvent fse)

■ 메소드 역할

- 서버는 클라이언트가 보낸 요청 파일 리스트에 응답하는 ack 이벤트 전송
- 현재 요청 파일 리스트에 대해 서버 측에서 온라인 모드 표시를 하는 등 처리 작업은 없음

■ 수정 방안

- ONLINE_MODE_LIST_ACK 이벤트 필드 설정 수정
 - Sender, receiver, requester 설정 삭제
 - 공통 필드 설정 (initiatorName, initiatorUuid, initiatorDeviceUuid)
- unicastEvent()에 uuid 파라미터 추가

■ 수정 구현

- String requester = ... 라인 삭제


```
...
```
- // create and send an ack event 아래 이벤트 필드 설정 및 전송 구문 다음으로 수정


```
CMFileSyncEventOnlineModeListAck ackEvent = CMFileSyncEventOnlineModeListAck();
// 공통 필드 설정 (sender, receiver, requester 대신)
ackEvent.setInitiatorName( listEvent.getInitiatorName() );
ackEvent.setInitiatorUuid( listEvent.getInitiatorUuid() );
ackEvent.setInitiatorDeviceUuid( listEvent.getInitiatorDeviceUuid() );
// 나머지 필드 설정
... (기존 코드 동일)
Boolean ret = CMEventManager.unicastEvent(ackEvent, listEvent.getInitiatorName(),
```



```
listEvent.getInitiatorUuid());
...
```

- Return true;

ONLINE_MODE_LIST_ACK 수신 및 END_ONLINE_MODE_LIST 송신

- CMFileSyncEventHandler

Private Boolean processONLINE_MODE_LIST_ACK(CMFileSyncEvent fse)

■ 메소드 역할

- 클라이언트 측에서 서버가 보낸 온라인 모드 전환 결과를 처리하는 메소드로, ACK 이벤트의 유효성을 검증하고 온라인 모드 요청 큐·경로 맵을 갱신한 뒤 필요한 경우 온라인 모드 전송 완료 이벤트를 다시 서버로 전송

■ 수정 방안

- CMFileSyncInfo 구하기 수정
- END_ONLINE_MODE_LIST 이벤트 필드 설정 수정
 - Sender, receiver, requester 설정 삭제
 - 공통 필드 설정 (initiatorName, initiatorUuid, initiatorDeviceUuid) 추가
- unicastEvent() 파라미터에 uuid 추가

■ 수정 구현

- // get online-mode-request queue 아래 라인 다음으로 수정
CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
...
- // check if the online-mode-request queue is empty 아래 if 문 블록을 다음으로 수정
if(requestQueue.isEmpty()) {
 // create and send the end-online-mode event
 CMFileSyncEventEndOnlineModeList endEvent = new
 CMFileSyncEventEndOnlineModeList();
 // 공통 필드 설정
 endEvent.setInitiatorName(ackEvent.getInitiatorName());
 endEvent.setInitiatorUuid(ackEvent.getInitiatorUuid());
 endEvent.setInitiatorDeviceUuid(ackEvent.getInitiatorDeviceUuid());
 // 나머지 필드 설정
 endEvent.setNumOnlineModeFiles(...); // 기존 코드 동일

 Boolean ret = CMEventManager.unicastEvent(endEvent, ackEvent.getSender(),
 ackEvent.getSenderUuid());
 ...
}

- Return true;

END_ONLINE_MODE_LIST 수신 및 END_ONLINE_MODE_LIST_ACK 송신

- CMFileSyncEventHandler

Private Boolean processEND_ONLINE_MODE_LIST(CMFileSyncEvent fse)

■ 메소드 역할

- 서버는 온라인 모드 요청 파일 리스트 끝이라는 이벤트를 수신하여, 특별히 하는 작업이 없기 때문에 return code 1 로 ack 이벤트 전송

■ 수정 방안

- END_ONLINE_MODE_LIST_ACK 이벤트 필드 수정
 - Sender, receiver, requester 설정 삭제
 - 공통 필드 설정 추가 (initiatorName, initiatorUuid, initiatorDeviceUuid)
- unicastEvent()에 uuid 파라미터 추가

■ 수정 구현

- // create and send ack event 아래 이벤트 필드 설정 및 전송 구문 다음으로 수정
 CMFileSyncEventEndOnlineModeListAck ackEvent = new
 CMFileSyncEventEndOnlineModeListAck();
 // 공통 필드 설정
 ackEvent.setInitiatorName(endEvent.getInitiatorName());
 ackEvent.setInitiatorUuid(endEvent.getInitiatorUuid());
 ackEvent.setInitiatorDeviceUuid(endEvent.getInitiatorDeviceUuid());
 // 나머지 필드 설정
 ackEvent.setNumOnlineModeFiles(endEvent.getNumOnlineModeFiles());
 ackEvent.setReturnCode(1);

 Boolean ret = CMEEventManager.unicastEvent(ackEvent, endEvent.getSender(),
 getSenderUuid());
 ...
- Return true;

END_ONLINE_MODE_LIST_ACK 수신

- CMFileSyncEventHandler

Private Boolean processEND_ONLINE_MODE_LIST_ACK(CMFileSyncEvent fse)

■ 메소드 역할

- 클라이언트는 서버로부터 온라인 모드 변경 요청 파일 리스트 전송 완료에 대한 ack 이벤트를 받고 마무리 작업 진행

- 온라인 모드 파일 및 크기 매핑 정보를 파일로 저장하고, 동기화 디렉토리 모니터링 재시작
- Sync() 한 번 수행 (필요한지 여부 재고려 필요)
- 수정 방안
 - CMFileSyncInfo 구하기 수정
- 수정 구현
 - CMFileSyncInfo syncInfo = ... 라인을 다음으로 수정
 CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
 ...

로컬 모드로 변경 과정에서의 이벤트 송수신 처리 수정

- 프로토콜
 - 시작 시점
 - CMFileSyncManager 의 requestLocalMode() 메소드 호출
 - 이 메소드는 클라이언트 app 에서 바로 호출하거나 파일 동기화 **proactive** 모드에서 디렉토리 활성화 정도에 따라 자동 호출
 - 사용 이벤트
 - LOCAL_MODE_LIST (client -> server)
 - LOCAL_MODE_LIST_ACK (server -> client)
 - END_LOCAL_MODE_LIST (client -> server)
 - END_LOCAL_MODE_LIST_ACK (server -> client)
 - 수정 방안
 - 사용 이벤트들에 공통인 requester 필드는 삭제하고 initiator 공통 필드 (initiatorName, initiatorUuid, initiatorDeviceUuid) 추가
 - 현재 파일 모드 변경의 initiator 는 클라이언트
 - 다중 로그인 상황에서 클라이언트들의 모드 변경 프로토콜은 별도 문서에서 설계 예정

LOCAL_MODE_LIST 송신

- CMFileSyncManager


```
Public Boolean requestLocalMode(List<Path> pathList)
```

■ 메소드 역할

- 선택한 동기화 파일들을 “로컬 모드”로 전환하도록 서버에 요청 이벤트를 보내고, 해당 요청을 추적하기 위해 내부 큐를 갱신
- 파일 전용 리스트를 이벤트 크기 한도에 맞춰 여러 덩어리로 나누어 `CMFileSyncEventLocalModeList` 이벤트를 반복적으로 생성·전송
- 모든 이벤트 전송이 성공하면, 파일 전용 리스트 전체를 `localModeRequestQueue`에 추가해 후속 처리를 위해 대기

■ 수정 방안

- `CMFileSyncInfo` 구하기 수정
- 변수명 `userName` 을 `initiatorName` 으로 수정
- 변수 `initiatorUuid`, `initiatorDeviceUuid` 추가
- 이벤트로 보낼 상대 경로 리스트 구하기 메소드 변경
 - `createSubLocalModeListForEvent()` -> `createSubPathListForEvent()` 위
`ONLINE_MODE_LIST` 송신 부분의 수정 구현 참조
- `LOCAL_MODE_LIST` 이벤트 필드 설정 수정
 - `Sender`, `receiver`, `requester` 설정 삭제
 - 공통 필드 설정 추가 (`initiatorName`, `initiatorUuid`, `initiatorDeviceUuid`)
- `unicastEvent()` 파라미터 `server uuid (null)` 추가

■ 수정 구현

- `CMFileSyncInfo syncInfo = ...` 라인 다음으로 수정
`CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();`
...
- `// get the user and server names` 아래 관련 구문 다음으로 수정
`CMInteractionInfo interInfo = CMInteractionInfo.getInstance();`
`String initiatorName = interInfo.getMyself().getName();`
`UUID initiatorUuid = interInfo.getMyself().getUuid();`
`UUID initiatorDeviceUuid = syncInfo.getDeviceUuid();`
`String serverName = interInfo.getDefaultServerInfo().getServerName();`
...
- `While(listIndex < fileOnlyList.size())` 블록 다음으로 수정
`List<Path> relativeFileOnlyList = toRelativePathList(fileOnlyList, getClientSyncHome());`

`while( listIndex < relativeFileOnlyList.size() ) {`

```

// create an event
CMFileSyncEventLocalModeList listEvent = new CMFileSyncEventLocalModeList();
// 공통 필드 설정
listEvent.setInitiatorName( initiatorName );
listEvent.setInitiatorUuid( initiatorUuid );
listEvent.setInitiatorDeviceUuid( initiatorDeviceUuid );

// get relative path list to be added to this event 아래 라인을 다음으로 수정
List<Path> subList = createSubPathListForEvent(listEvent.getBytesNum(), relativeFileOnlyList,
listIndex);

// send the event
sendResult = CMEventManager.unicastEvent(listEvent, serverName, null);
...
}
...

```

- Return true;

LOCAL_MODE_LIST 수신 및 LOCAL_MODE_LIST_ACK 송신

- CMFileSyncEventHandler

private Boolean processLOCAL_MODE_LIST(CMFileSyncEvent fse)

■ 메소드 역할

- 클라이언트가 로컬 모드를 요청한 파일 목록에 대해 파일 전송을 시작하고 ack 이벤트 전송

■ 수정 방안

- 변수명 수정
 - Requester -> initiator
- CMFileTransferManager.pushFile() 메소드의 파라미터로 수신자 uuid 추가 (수신 이벤트의 senderUuid 나 initiatorUuid)
- LOCAL_MODE_LIST_ACK 이벤트 필드 설정 수정
 - Sender, receiver, requester 설정 삭제
 - 공통 필드 설정 (initiatorName, initiatorUuid, initiatorDeviceUuid)
- unicastEvent()에 uuid 파라미터 추가

■ 수정 구현

- String requester = ... 라인을 다음으로 수정
 String initiatorName = listEvent.getInitiatorName();
 UUID initiatorUuid = listEvent.getInitiatorUuid();

```
UUID initiatorDeviceUuid = listEvent.getInitiatorDeviceUuid();
```

```
... (이후 requester 부분을 initiatorName 으로 수정)
```

- Path serverSyncHome = ... 라인을 다음으로 수정
Path serverSyncHome = syncManager.getServerSyncHome(initiatorName);
...
- Ret &= CMFileTransferManager.pushFile(...) 라인을 다음으로 수정
ret &= CMFileTransferManager.pushFile(absPath.toString(), initiatorName, initiatorUuid);
...
- // create and send ack event 아래 이벤트 설정 및 송신 부분 다음으로 수정
CMFileSyncEventLocalModeListAck ackEvent = new CMFileSyncEventLocalModeList();
// sender, receiver, requester 설정 삭제
// 공통 필드 설정 추가
ackEvent.setInitiatorName(initiatorName);
ackEvent.setInitiatorUuid(initiatorUuid);
ackEvent.setInitiatorDeviceUuid(initiatorDeviceUuid);
// 나머지 필드 설정 기존 코드 동일 (relative path list, return code)
...
- Ret = CMEventManager.unicastEvent(...) 라인을 다음으로 수정
ret = CMEventManager.unicastEvent(ackEvent, initiatorName, initiatorUuid);
...
- Return true;

LOCAL_MODE_LIST_ACK 수신

- CMFileSyncEventHandler
private Boolean processLOCAL_MODE_LIST_ACK(CMFileSyncEvent fse)
 - 메소드 역할
 - 서버가 보낸 ack 이벤트 수신하여 return code 디버그용으로 출력
 - 수정 방안
 - 수정 사항 없음

클라이언트측 파일 수신 완료 후 로컬 모드 변경 완료 여부 체크 처리 및 END_LOCAL_MODE_LIST 송신

- CMFileSyncManager
public void checkTransferForLocalMode(CMFileEvent fe)
 - 메소드 역할
 - 파일 수신자가 파일 수신 완료 때마다 마지막으로 이 메소드를 호출하여 로컬 모드 변경용
파일 수신이 완료되었는지 여부 확인

- 클라이언트에서 수신한 파일이 로컬 모드 요청 큐의 예상 항목과 일치하는지 확인하고, 파일을 최종 위치로 이동한 뒤 동기화 상태를 갱신하며 필요 시 종료 이벤트를 서버로 전송

■ 수정 방안

- Path transferFileHome 구하기 변경 (CM conf info 구하기 변경)
- CMFileSyncInfo 구하기 변경
- END_LOCAL_MODE_LIST 이벤트 필드 설정 변경
 - Sender, receiver, requester 필드 설정 삭제
 - 공통 필드 (initiatorName, initiatorUuid, initiatorDeviceUuid) 설정 추가
 - numLocalModeFiles 필드 설정 유지
- unicastEvent() 파라미터에 fileSenderId 추가

■ 수정 구현

- 변수 추가 (fileSender 아래 fileSenderId 추가)


```
...
UUID fileSenderId = fe.getFileSenderId();
...
```
- Path transferFileHome = ... 라인을 다음으로 수정


```
CMConfigurationInfo confInfo = CMConfigurationInfo.getInstance();
Path transferFileHome = confInfo.getTransferredFileHome();
...
```
- CMFileSyncInfo syncInfo = ... 라인을 다음으로 수정


```
CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
...
```
- // if the queue is empty, create and send an end-local-mode-list event 아래 이벤트 필드 설정 부분 다음으로 수정


```
CMFileSyncEventEndLocalModeList endEvent = new CMFileSyncEventEndLocalModeList();
// sender, receiver, requester 필드 설정 삭제
// 공통 필드 설정 추가 (initiatorName, initiatorUuid, initiatorDeviceUuid, 클라이언트
자신의 정보 또는 파일 수신자 정보)
endEvent.setInitiatorName( fe.getFileReceiver() );
endEvent.setInitiatorUuid( fe.getFileReceiverUuid() );
endEvent.setInitiatorDeviceUuid( syncInfo.getDeviceUuid() );
// filter only file type from the path list 아래 나머지 필드 설정 기존 코드 동일
...
```

- unicastEvent(...) 라인 다음으로 수정
 Boolean ret = CMEventManager.unicastEvent(endEvent, fileSender, fileSenderUuid);
 ...
- return;

END_LOCAL_MODE_LIST 수신 및 END_LOCAL_MODE_LIST_ACK 송신

- CMFileSyncEventHandler

private Boolean processEND_LOCAL_MODE_LIST(CMFileSyncEvent fse)

■ 메소드 역할

- 서버가 클라이언트로부터 모든 요청 파일에 대해 로컬 모드 변경 완료되었다는 이벤트 수신하고, 이에 대한 ack 이벤트 다시 전송

■ 수정 방안

- END_LOCAL_MODE_LIST_ACK 이벤트 필드 설정 수정
 - Sender, receiver, requester 필드 설정 삭제
 - 공통 필드 설정 추가 (initiatorName, initiatorUuid, initiatorDeviceUuid)
- unicastEvent() 메소드에 수신자 uuid 파라미터 추가

■ 수정 구현

- // create an end-local-mode-list ack event 로 코멘트 수정하고 아래 이벤트 필드 설정 부분 다음으로 수정
 CMFileSyncEventEndLocalModeListAck ackEvent = new
 CMFileSyncEventEndLocalModeListAck();
 // sender, receiver, requester 필드 설정 삭제
 // 공통 필드 설정 추가 (initiatorName, initiatorUuid, initiatorDeviceUuid)
 ackEvent.setInitiatorName(endEvent.getInitiatorName());
 ackEvent.setInitiatorUuid(endEvent.getInitiatorUuid());
 ackEvent.setInitiatorDeviceUuid(endEvent.getInitiatorDeviceUuid());
 // 나머지 필드 설정 기존 코드 동일 (numLocalModeFiles, returnCode)
 ...
- unicastEvent(...) 라인 다음으로 수정
 Boolean ret = CMEventManager.unicastEvent(ackEvent, endEvent.getSender(),
 endEvent.getSenderUuid());
 ...
- return true;

END_LOCAL_MODEL_LIST_ACK 수신

- CMFileSyncEventHandler

private Boolean processEND_LOCAL_MODE_LIST_ACK(CMFileSyncEvent fse)

- 메소드 역할
 - 클라이언트는 서버로부터 로컬 모드 변경 완료에 대한 **ack** 이벤트를 수신하여 마지막 처리 작업 (동기화 후속 작업) 진행
 - 서버로부터 로컬 모드 파일 정보 전송이 정상적으로 마무리되었음을 확인하고, 이후 지속적인 파일 감시와 동기화를 위한 준비를 완료
- 수정 방안
 - CMFileSyncInfo 구하기 수정
 - 그 외 수정 사항 없음
- 수정 구현
 - CMFileSyncInfo syncInfo = ... 라인을 다음으로 수정
 CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
 ...

서버 측 메타 정보 파일들 캐싱하는 매핑 클래스 설계

- 서버 측 파일 동기화 관련 메타 정보 파일에 파일 모드 변경 관련 **op** 관리 방안
 - 파일 모드 변경은 파일 메타 정보 변경하지 않으므로 여기서 관리 안함
 - ~~■ 파일별 엔트리 정보에 온라인 모드인지 여부 확인하는 Boolean 타입의 isOnline 추가~~
 - ~~■ changeLog의 OP에 CHANGEMODE 추가~~
 - 내부적으로 isOnline 값 수정
- 서버 측 파일 동기화 관련 메타 정보 파일
 - index.json (디바이스별)
 - device index: 해당 디바이스의 현재 파일 상태 스냅샷
 - 파일 저장 디렉토리
 - CM 설정 디렉토리 (.cm-settings) → file-sync → server → client name → device uuid
 - 내용
 - schema (?)
 - generatedAt
 - lastChangeId : 마지막 반영 변경 id (cursor 와 동일)
 - entries
 - path
 - isDirectory
 - contentHash

- mtime
 - size
 - ~~■ isOnline~~
 - tombstone
 - version
 - lastChangeld
- cursor (디바이스별)
 - 마지막 반영 변경 id
 - 파일 저장 디렉토리
 - CM 설정 디렉토리 (.cm-settings) → file-sync → server → client name → device uuid
 - 내용
 - 마지막 반영 id 숫자값 한줄
- changelog-yyyy-mm-dd.jsonl (클라이언트별)
 - ChangeLog: 사용자별 파일 동기화 변경 로그 기록 (일별 파일 생성)
 - 파일 저장 디렉토리
 - CM 설정 디렉토리 (.cm-settings) → file-sync → server → client name
 - 내용
 - changeld: 사용자별 전역 단조 증가 번호
 - userName: 사용자 이름
 - originDeviceUuid: 변경을 발생시킨 디바이스
 - op: 작업 종류 (CREATE, DELETE, MODIFY, ~~RENAME, UPSERT, CHANGEMODE~~)
 - path: 대상 파일 경로 (파일 동기화 홈 제외한 상대 경로)
 - isDirectory: 대상 파일 디렉토리 여부
 - ~~renameFrom~~: RENAME 시 원래 경로 (없으면 생략)
 - contentHash: Strong hash (RENAME 시 동일 hash 유지)
 - mtime: 수정 시각 (초 단위 epoch)
 - size: 파일 크기 (바이트)
 - ~~● isOnline: 온라인 모드인지 여부~~
 - tombstone: 삭제 여부 (true 면 삭제 상태)
 - ts: 서버가 로그 기록한 시각 (ISO 8601)
- 메모리 매핑 클래스 구조
 - 데이터 모델 (불변 VO)
 - Public record CMFileSyncIndexEntry (파일 엔트리용)
 - Public record CMFileSyncIndexSnapshot (인덱스 파일의 불변 스냅샷)
 - 인메모리 인덱스 (단일 writer)
 - Public class CMFileSyncInMemoryIndex (인덱스 파일용)

- 스냅샷 I/O
 - Public interface CMFileSyncIndexSnapshotStore
 - Public class CMFileSyncJacksonSnapshotStore (구현체)
- 리포지토리 (한 디바이스 단위)
 - Public class CMFileSyncIndexRepository (인덱스 파일, 스냅샷스토어)
- 사용자, 디바이스별 스냅샷 관리 Map
 - Public record CMFileSyncStateKey
 - Public class CMFileSyncIndexRegistry

CMFILESYNCEXENTRY

- 코드

```
public record CMFileSyncIndexEntry(
    String path,
    Boolean isDirectory,
    String contentHash,
    long mtime,
    long size,
    Boolean isOnline,
    boolean tombstone,
    int version,
    long lastChangeld
) {}
```

CMFILESYNCEXSNAPSHOT

- 코드

```
public record CMFileSyncIndexSnapshot(
    long lastChangeld,
    Map<String, CMFileSyncIndexEntry> entries, // 불변 Map
    String generatedAt
) {}
```

CMFILESYNCEXMEMORYINDEX

- 코드

```
public class CMFileSyncInMemoryIndex {
    private volatile long lastChangeld = 0L;
    private final HashMap<String, CMFileSyncIndexEntry> table = new HashMap<>();

    // --- 단일 writer 경로 ---
    public void upsert(CMFileSyncIndexEntry e) {
        table.put(e.path(), e);
        lastChangeld = Math.max(lastChangeld, e.lastChangeld());
    }
}
```

```

}

public void delete(String path, long changeld, long now) {
    CMFileSyncIndexEntry prev = table.get(path);
    int v = prev == null ? 1 : prev.version() + 1;
    table.put(path, new CMFileSyncIndexEntry(path, prev.isDirectory(),
        prev == null ? null : prev.contentHash(), now, 0L, prev.isOnline(), true, v, changeld));
    lastChangeld = Math.max(lastChangeld, changeld);
}

public void rename(String from, String to, CMFileSyncIndexEntry newMeta) {
    CMFileSyncIndexEntry prev = table.get(from);
    int vOld = prev == null ? 1 : prev.version() + 1;
    table.put(from, new CMFileSyncIndexEntry(from, prev.isDirectory(),
        prev == null ? null : prev.contentHash(), newMeta.mtime(),
        0L, prev.isOnline(), true, vOld, newMeta.lastChangeld()));
    upsert(newMeta);
}

// --- 다중 reader: 스냅샷 제공 ---
public CMFileSyncIndexSnapshot snapshot() {
    return new CMFileSyncIndexSnapshot(
        lastChangeld, Map.copyOf(table),
        java.time.OffsetDateTime.now().toString());
}

public CMFileSyncIndexEntry get(String path) { return table.get(path); }

public long lastChangeld() { return lastChangeld; }
}

```

CMFILESYNINDEXSNAPSHOTSTORE

- 코드

```

public interface CMFileSyncIndexSnapshotStore {
    CMFileSyncIndexSnapshot loadOrEmpty(java.nio.file.Path dir) throws java.io.IOException;
    void saveAtomically(java.nio.file.Path dir, CMFileSyncIndexSnapshot snap) throws java.io.IOException;
}

```

CMFILESYNJACKSONSNAPSHOTSTORE

- 코드

```

package your.package.cmfilesync;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.SerializationFeature;
import com.fasterxml.jackson.databind.DeserializationFeature;
import com.fasterxml.jackson.core.type.TypeReference;

import java.io.IOException;
import java.nio.file.*;
import java.util.Map;

```

```

public class CMFileSyncJacksonSnapshotStore implements CMFileSyncIndexSnapshotStore {

    private final ObjectMapper mapper;

    public CMFileSyncJacksonSnapshotStore() {
        mapper = new ObjectMapper()
            .enable(SerializationFeature.INDENT_OUTPUT)
            .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)
            .disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES);
    }

    @Override
    public CMFileSyncIndexSnapshot loadOrEmpty(Path dir) throws IOException {
        Path file = dir.resolve("index.json");
        if (!Files.exists(file)) {
            // 빈 스냅샷 반환
            return new CMFileSyncIndexSnapshot(0L, Map.of(), nowString());
        }
        return mapper.readValue(file.toFile(), CMFileSyncIndexSnapshot.class);
    }

    @Override
    public void saveAtomically(Path dir, CMFileSyncIndexSnapshot snap) throws IOException {
        if (!Files.exists(dir)) {
            Files.createDirectories(dir);
        }

        Path tmp = dir.resolve("index.json.tmp");
        Path target = dir.resolve("index.json");

        // JSON 직렬화 후 tmp 파일에 쓰기
        mapper.writeValue(tmp.toFile(), snap);

        // fsync 보장
        try (var channel = FileChannel.open(tmp, StandardOpenOption.READ)) {
            channel.force(true);
        }

        // 원자적 교체
        Files.move(tmp, target, StandardCopyOption.ATOMIC_MOVE, StandardCopyOption.REPLACE_EXISTING);

        // 디렉토리 fsync
        try (var dirChannel = FileChannel.open(dir, StandardOpenOption.READ)) {
            dirChannel.force(true);
        }
    }

    private String nowString() {
        return java.time.OffsetDateTime.now().toString();
    }
}

```

CMFILESYNINDEXREPOSITORY

- 코드

```

public class CMFileSyncIndexRepository {
    private final CMFileSyncInMemoryIndex mem = new CMFileSyncInMemoryIndex();
    private final CMFileSyncIndexSnapshotStore store;
    private final java.nio.file.Path dir; // /srv/.../<user>/<device>/

    public CMFileSyncIndexRepository(CMFileSyncIndexSnapshotStore store, java.nio.file.Path dir) {
        this.store = store; this.dir = dir;
    }

    public void warmUpFromDisk() throws java.io.IOException {
        var snap = store.loadOrEmpty(dir);
        snap.entries().values().forEach(mem::upsert);
    }

    // ChangeLog 적용 경로(단일 writer 에서만 호출)
    public void applyCreateOrModify(String path, Boolean isDirectory, String hash, long mtime, long size,
boolean isOnline, long changeld) {
        var prev = mem.get(path);
        int v = prev == null ? 1 : prev.version() + 1;
        mem.upsert(new CMFileSyncIndexEntry(path, isDirectory, hash, mtime, size, isOnline, false, v, changeld));
    }

    public void applyDelete(String path, long changeld, long now) {
        mem.delete(path, changeld, now);
    }

    public void applyRename(String from, String to, boolean isDirectory, String hash, long mtime, long size,
boolean isOnline, long changeld) {
        var prevTo = mem.get(to);
        int v = prevTo == null ? 1 : prevTo.version() + 1;
        var meta = new CMFileSyncIndexEntry(to, isDirectory, hash, mtime, size, isOnline, false, v, changeld);
        mem.rename(from, to, meta);
    }

// 파일 모드 변경 시에도 createOrModify() 메소드 그냥 사용해도 될듯..
public void applyChangeMode( String path, Boolean isDirectory, String hash, long mtime, long size, boolean
isOnline, long changeld ) {
—var prev = mem.get(path);
—int v = prev == null ? 1 : prev.version() + 1;
—mem.upsert(new CMFileSyncIndexEntry(path, isDirectory, hash, mtime, size, isOnline, false, v, changeld));

    }

    public void flushSnapshot() throws java.io.IOException {
        store.saveAtomically(dir, mem.snapshot());
    }

    // 읽기용
    public CMFileSyncIndexEntry readEntry(String path) { return mem.get(path); }
    public CMFileSyncIndexSnapshot readSnapshot() { return mem.snapshot(); }
    public long lastChangeld() { return mem.lastChangeld(); }
}

```

CMFILESYNCSSTATEKEY

● 코드

```

public record CMFileSyncStateKey(String userId, String deviceId) {

```

```
@Override public String toString() { return userId + "/" + deviceId; }
}
```

CMFILESYNCINDEXREGISTRY

- 코드

```
public class CMFileSyncIndexRegistry {
    private final ConcurrentHashMap<CMFileSyncStateKey, CMFileSyncIndexRepository> repos
        = new ConcurrentHashMap<>();
    private final CMFileSyncIndexSnapshotStore store;
    private final Path baseDir; // /srv/yourapp/index/

    public CMFileSyncIndexRegistry(CMFileSyncIndexSnapshotStore store, Path baseDir) {
        this.store = store; this.baseDir = baseDir;
    }

    public CMFileSyncIndexRepository getOrLoad(String userId, String deviceId) {
        var key = new CMFileSyncStateKey(userId, deviceId);
        return repos.computeIfAbsent(key, k -> {
            var dir = baseDir.resolve(userId).resolve(deviceId);
            var repo = new CMFileSyncIndexRepository(store, dir);
            try { repo.warmUpFromDisk(); } catch (Exception ignore) {}
            return repo;
        });
    }

    public void flush(String userId, String deviceId) {
        var repo = repos.get(new CMFileSyncStateKey(userId, deviceId));
        if (repo == null) return;
        try { repo.flushSnapshot(); } catch (Exception e) { /* log */ }
    }

    // 선택: idle 기준으로 플러시+제거
    public void evict(java.util.function.BiPredicate<CMFileSyncStateKey, CMFileSyncIndexRepository>
        shouldEvict) {
        repos.forEach((k, r) -> {
            if (shouldEvict.test(k, r)) {
                flush(k.userId(), k.deviceId());
                repos.remove(k);
            }
        });
    }
}
```

CMFILESYNCENTRY

- 수정 방안

- **CMFileSyncIndexEntry** 로 대체 가능 여부 확인하고 가능하면 교체
- 이 클래스 내에서 정의된 파일 엔트리 메타 정보는 위에서 새로 정의한 **CMFileSyncIndexRepository** 에서 가져올 수 있음

- 이를 위한 hydrate() 메소드 추가 가능
- 적용 멤버 변수
 - Path pathRelativeToHome, long size, FileTime lastModifiedTime;
 - FileTime <-> epoch seconds (Instant) 변환 필요
- 수정 예시 코드

```
class CMFileSyncEntry {
    final String path;
    boolean isDirectory;
    long size;
    long mtime;
    String hash;
Boolean isOnline;
    boolean tombstone;
    long sourceChangeld;

    CMFileSyncEntry(String path) { this.path = path; }

    void hydrate(CMFileSyncIndexRepository repo) {
        var e = repo.readEntry(path);
        if (e != null) {
            size = e.size(); mtime = e.mtime(); hash = e.contentHash();
            tombstone = e.tombstone(); sourceChangeld = e.lastChangeld();
        }
    }
}
```

CMFILESYNCFIELD

- 수정 방안
 - 멤버 변수로 CMFileSyncIndexRegistry 추가 및 초기화
- 수정 코드

```
public class CMFileSyncInfo {
    // 기존 CMFileSyncInfo 필드들...

    // 새로 추가: user/device 별 인덱스 스냅샷 관리
    private final CMFileSyncIndexRegistry indexRegistry;

    public CMFileSyncInfo() {
        if (confInfo.getSystemType().equals("SERVER")) {
            Path baseIndexDir = Path.of(".cm-settings/file-sync/server");
            CMFileSyncSnapshotStore = new CMFileSyncJacksonSnapshotStore();
            this.indexRegistry = new CMFileSyncIndexRegistry(store, baseIndexDir);
        }
        ...
    }

    public CMFileSyncIndexRegistry getIndexRegistry() {
```



```

    return indexRegistry;
}

// 필요 시 초기화, 플러시, 종료 처리
public void flushAllIndexes() {
    // idle 또는 서버 종료 시 모든 인덱스 스냅샷 저장
    // 예: indexRegistry.evict(...)
}
}

```

서버 측 메타 정보 파일 업데이트 및 클라이언트 전달 방안 설계

- 서버 측 메타 파일 공통 원칙: “변경 수신 완료” 시점에 서버 상태 반영
 - (1) changelog append → (2) in-memory index apply → (3) 디바이스 cursor 갱신 → (4) 마지막으로 index flush (이벤트 1건 아닌 배치 단위로 모아 호출 권장)
 - index.json
 - 서버가 실제 파일 변경을 성공적으로 반영한 직후 인메모리 인덱스에 적용
 - 배치 기준 시점에 파일로 flush
 - cursor
 - 해당 디바이스에서 유래한 변경을 서버가 성공 반영하고, cursor 값을 changeld 로 갱신 후에 ack 이나 관련 이벤트 클라이언트로 전송
 - 클라이언트로 보내는 이벤트에 cursor 필드 추가
 - changelog-YYYY-MM-DD.jsonl
 - 서버가 “쓰기 성격”의 이벤트를 커밋하는 순간 단조 증가 changeld 로 한 줄 append
- 파일 op 별 메타 파일 수정안
 - 구현 참고
 - <https://chatgpt.com/share/68a6ffe2-ed58-800f-a28f-0e2eb2c396>
 - 신규 파일 추가
 - 완료 시점
 - CMFileSyncManager.completeNewFileTransfer()
 - 메타 파일 업데이트 방안
 - index.json: 새 엔트리 추가
 - path, isDirectory, contentHash, mtime, size, isOnline, tombstone=false, version=1
 - lastChangeld=<증가값>
 - cursor: lastChangeld 숫자 1 줄 갱신
 - changelog
 - 위에서 정의한 양식으로 append (op = CREATE)
 - 파일 수정
 - 완료 시점
 - CMFileSyncManager.completeUpdateFile()
 - 메타 파일 업데이트 방안

- index.json: 해당 파일 엔트리 갱신
 - path, **isDirectory**, contentHash, mtime, size, **isOnline**, tombstone=false, version+=1
 - lastChanged 증가
 - cursor: 증가된 lastChangeId 값으로 덮어쓰기
 - changelog
 - 위에서 정의한 양식으로 append (op = MODIFY)
- 파일 삭제
 - 완료 시점
 - CMFileSyncGenerator.deleteFilesAndUpdateBasisFileList()
 - 별도의 삭제 이벤트는 현재 없음 (프로토콜 개선 필요) -> COMPLETE_DELETE_FILE 이벤트 생성하여 전송 (다음 장에서 별도 설계)
 - 메타 파일 업데이트 방안
 - index.json
 - path, **isDirectory**, (contentHash, mtime, size,) **isOnline**, tombstone=true, version+=1
 - lastChangeId 증가
 - cursor: 증가된 lastChangeId 로 갱신
 - changelog
 - 위에서 정의한 양식으로 append (op = DELETE)
- ~~■ 파일 모드 온라인으로 변경~~
 - ~~● 완료 시점~~
 - ~~● CMFileSyncEventHandler.processEND_ONLINE_MODE_LIST()~~
 - ~~● 메타 파일 업데이트 방안~~
 - ~~● index.json: 해당 파일 엔트리 갱신~~
 - ~~■ path, **isDirectory**, contentHash, mtime, size, **isOnline**, tombstone=false, version+=1~~
 - ~~■ lastChanged 증가~~
 - ~~● cursor: 증가된 lastChangeId 값으로 덮어쓰기~~
 - ~~● changelog~~
 - ~~● 위에서 정의한 양식으로 append (op = CHANGEMODE)~~
- ~~■ 파일 모드 로컬로 변경~~
 - ~~● 완료 시점~~
 - ~~● CMFileSyncEventHandler.processEND_LOCAL_MODE_LIST()~~
 - ~~● 메타 파일 업데이트 방안~~
 - ~~● index.json: 해당 파일 엔트리 갱신~~
 - ~~■ path, **isDirectory**, contentHash, mtime, size, **isOnline**, tombstone=false, version+=1~~
 - ~~■ lastChanged 증가~~
 - ~~● cursor: 증가된 lastChangeId 값으로 덮어쓰기~~
 - ~~● changelog~~
 - ~~● 위에서 정의한 양식으로 append (op = CHANGEMODE)~~
- 디렉토리 op 별 메타 파일 수정안
 - path 에서 디렉토리 여부에 대한 메타 파일 정보 추가 (위쪽에 수정 반영)
 - index
 - path 다음에 isDirectory 항목 추가 (Boolean)
 - Changelog

- Path 다음에 isDirectory 항목 추가
- 인메모리 클래스
 - 엔트리에 해당하는 클래스(CMFileSyncIndexEntry)에 멤버 변수 추가
 - Boolean isDirectory;
 - 엔트리 추가, 수정, 삭제 관련 메소드에 isDirectory 파라미터 및 추가 처리
- 신규 디렉토리 추가
 - 추가 완료 시점
 - 클라이언트로부터 파일 목록 수신 후에
CMFileSyncGenerator.requestTransferOfNewFiles()에서
CMFileSyncManager.completenewFileTransfer() 호출 (파일 신규 추가와 동일 시점)
- 디렉토리 삭제
 - 삭제 완료 시점
 - 클라이언트로부터 파일 목록 수신 후에
CMFileSyncGenerator.deleteFilesAndUpdateBasisFileList()에서 삭제

CMUTIL

- 멤버 메소드 추가 (md5 strong hash 구하는 메소드를 CMFileSyncManager 에서 이전하고 새로 추가)

```
// CM/src/main/java/kr/ac/konkuk/ccslab/cm/util/CMUtil.java (추가 메소드)
import java.nio.file.Files;
import java.nio.file.Path;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

public class CMUtil {
    // ... 기존 내용 유지

    public static byte[] md5(Path path) throws IOException {
        MessageDigest md;
        try {
            md = MessageDigest.getInstance("MD5");
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException("MD5 not available", e);
        }
        try (InputStream in = Files.newInputStream(path)) {
            byte[] buf = new byte[8192];
            int n;
            while ((n = in.read(buf)) > 0) md.update(buf, 0, n);
        }
        return md.digest();
    }

    public static String md5Hex(Path path) throws IOException {
        byte[] digest = md5(path);
        StringBuilder sb = new StringBuilder(digest.length * 2);
        for (byte b : digest) {
            int v = b & 0xFF;
            if (v < 16) sb.append('0');
            sb.append(Integer.toHexString(v));
        }
    }
}
```

```

        return sb.toString();
    }
}

```

- CMFileSyncManager 의 calculateFileChecksum() 메소드는 코멘트 처리
 - 기존 이 메소드 호출하던 곳을 (2 군데) CMUtil.md5() 메소드로 변경

CMFILESYNCFINFO

- 멤버 메소드 추가
 - 파일 추가 (CREATE) op 완료에 대한 메모리 인덱스 및 메타 파일 정보 업데이트
 - 구현 코드

```

// 서버에서 Path 로부터 hash, mtime, size 직접 구하기
public void applyCreate(String userId, String deviceId, String path) throws IOException {
    CMFileSyncIndexRepository repo = getIndexRegistry().getOrLoad(userId, deviceId);
    long newChangeld = repo.lastChangeld() + 1;

    // CMFileSyncManager 인스턴스 얻기
    CMFileSyncManager syncManager = CMInfo.getInstance()
        .getServiceManager(CMFileSyncManager.class);

    // syncHome 얻기
    Path syncHome = syncManager.getServerSyncHome(userId);

    // --- 경로 정규화: abs(절대) / path(상대; 메타 기록용) ---
    Path input = Path.of(path);
    Path abs, relPath;
    if (input.isAbsolute()) {
        abs = input.toAbsolutePath().normalize();
        relPath = syncHome.relativize(abs).normalize();
        path = relPath.toString().replace("\\", '/'); // 메타에는 항상 상대경로 + '/'
    } else {
        relPath = input.normalize();
        abs = syncHome.resolve(relPath).toAbsolutePath().normalize();
        path = relPath.toString().replace("\\", '/');
    }

    Boolean isDirectory = Files.isDirectory(abs);
    String md5Hex = CMUtil.md5Hex(abs);
    long mtimeSec = Files.getLastModifiedTime(abs).toMillis() / 1000;
    long sizeBytes = Files.size(abs);

    repo.applyCreateOrModify(path, isDirectory, md5Hex, mtimeSec, sizeBytes, false, newChangeld);
    writeCursor(userId, deviceId, newChangeld);
    appendChangelog(userId, deviceId, "CREATE", path, isDirectory, md5Hex, mtimeSec, sizeBytes,
false, newChangeld);
    repo.flushSnapshot();
}

public void applyCreateFast(String userId, String deviceId,
    String path, Boolean isDirectory, String contentHash, long mtimeEpochSec, long
sizeBytes, Boolean isOnline) throws IOException {

```

```

        CMFileSyncIndexRepository repo = indexRegistry.getOrLoad(userId, deviceId); // warm-up
포함 :contentReference[oaicite:6]{index=6}
        long newChangeld = repo.lastChangeld() + 1; // changeld 는 서버가 단조증가로 부여

        // 1) 인메모리 인덱스 반영
        repo.applyCreateOrModify(path, isDirectory, contentHash, mtimeEpochSec, sizeBytes, isOnline,
newChangeld);

        // 2) cursor 파일 갱신
        writeCursor(userId, deviceId, newChangeld);

        // 3) changelog append
        appendChangelog(userId, deviceId, "CREATE", path, isDirectory, contentHash, mtimeEpochSec,
sizeBytes, isOnline, newChangeld);

        // 4) index 스냅샷 flush
        repo.flushSnapshot(); // 09 문서의 flush 설계에 부합 :contentReference[oaicite:7]{index=7}
    }

```

● 멤버 메소드 추가

- 파일 수정 (MODIFY) op 완료에 대한 메모리 인덱스 및 메타 파일 정보 업데이트
- 구현 코드

```

// 서버에서 Path 로부터 직접 hash, mtime, size 구하기
public void applyModify(String userId, String deviceId, String path) throws IOException {
    CMFileSyncIndexRepository repo = getIndexRegistry().getOrLoad(userId, deviceId);
    long newChangeld = repo.lastChangeld() + 1;

    CMFileSyncManager syncManager = CMInfo.getInstance()
        .getServiceManager(CMFileSyncManager.class);

    // syncHome 얻기
    Path syncHome = syncManager.getServerSyncHome(userId);

    // --- 경로 정규화: abs(절대) / path(상대; 메타 기록용) ---
    Path input = Path.of(path);
    Path abs, relPath;
    if (input.isAbsolute()) {
        abs = input.toAbsolutePath().normalize();
        relPath = syncHome.relativize(abs).normalize();
        path = relPath.toString().replace("\\", '/'); // 메타에는 항상 상대경로 + '/'
    } else {
        relPath = input.normalize();
        abs = syncHome.resolve(relPath).toAbsolutePath().normalize();
        path = relPath.toString().replace("\\", '/');
    }

    Boolean isDirectory = Files.isDirectory(abs);
    String md5Hex = CMUtil.md5Hex(abs);
    long mtimeSec = Files.getLastModifiedTime(abs).toMillis() / 1000;
    long sizeBytes = Files.size(abs);
    // get isOnline from previous info
    CMFileSyncIndexEntry prevEntry = repo.readEntry(path);
    Boolean isOnline = prevEntry == null ? false : prev.isOnline();

```

```

        repo.applyCreateOrModify(path, isDirectory, md5Hex, mtimeSec, sizeBytes, isOnline,
newChangeld);
        writeCursor(userId, deviceId, newChangeld);
        appendChangelog(userId, deviceId, "MODIFY", path, isDirectory, md5Hex, mtimeSec,
sizeBytes, isOnline, newChangeld);
        repo.flushSnapshot();
    }

    public void applyModifyFast(String userId, String deviceId,
        String path, Boolean isDirectory, String contentHash, long mtimeEpochSec, long
sizeBytes, Boolean isOnline) throws IOException {

        CMFileSyncIndexRepository repo = indexRegistry.getOrLoad(userId, deviceId);
        long newChangeld = repo.lastChangeld() + 1;

        repo.applyCreateOrModify(path, isDirectory, contentHash, mtimeEpochSec, sizeBytes,
isOnline, newChangeld);

        writeCursor(userId, deviceId, newChangeld);

        appendChangelog(userId, deviceId, "MODIFY", path, isDirectory, contentHash, mtimeEpochSec,
sizeBytes, isOnline, newChangeld);

        repo.flushSnapshot();
    }

```

- 멤버 메소드 추가

- 파일 삭제 (DELETE) op 완료에 대한 메모리 인덱스 및 메타 파일 정보 업데이트
- 구현 코드

```

public void applyDelete(String userId, String deviceId, String path) throws IOException {
    CMFileSyncIndexRepository repo = getIndexRegistry().getOrLoad(userId, deviceId);
    long newChangeld = repo.lastChangeld() + 1;

    CMFileSyncManager syncManager = CMInfo.getInstance()
        .getServiceManager(CMFileSyncManager.class);

    // syncHome 얻기
    Path syncHome = syncManager.getServerSyncHome(userId);

    // --- 경로 정규화: abs(절대) / path(상대; 메타 기록용) ---
    Path input = Path.of(path);
    Path abs, relPath;
    if (input.isAbsolute()) {
        abs = input.toAbsolutePath().normalize();
        relPath = syncHome.relativize(abs).normalize();
        path = relPath.toString().replace("\\", '/'); // 메타에는 항상 상대경로 + '/'
    } else {
        relPath = input.normalize();
        abs = syncHome.resolve(relPath).toAbsolutePath().normalize();
        path = relPath.toString().replace("\\", '/');
    }

    Boolean isDirectory = Files.isDirectory(abs);

```

```

        long nowSec = System.currentTimeMillis() / 1000;
        CMFileSyncIndexEntry prevEntry = repo.readEntry(path); // should not be null
        Boolean isOnline = prevEntry.isOnline();

        repo.applyDelete(path, newChangeld, nowSec);
        writeCursor(userId, deviceId, newChangeld);
        appendChangelog(userId, deviceId, "DELETE", path, isDirectory, null, nowSec, 0L, isOnline,
newChangeld);
        repo.flushSnapshot();
    }

```

● 멤버 메소드 추가

■ 파일 모드 변경 완료에 대한 메모리 인덱스 및 메타 파일 정보 업데이트

■ 구현 코드

```

public void applyChangeMode(String userId, String deviceId, String path, Boolean isOnline) throws
IOException {
    CMFileSyncIndexRepository repo = getIndexRegistry().getOrLoad(userId, deviceId);
    long newChangeld = repo.lastChangeld() + 1;

    CMFileSyncManager syncManager = CMInfo.getInstance()
        .getServiceManager(CMFileSyncManager.class);

    // 경로 정규화: abs(절대) / path(상대; 메타 기록용)
    Path input = Path.of(path);
    Path abs, relPath;
    if (input.isAbsolute()) {
        abs = input.toAbsolutePath().normalize();
        relPath = syncHome.relativize(abs).normalize();
        path = relPath.toString().replace("\\", "/"); // 메타에는 항상 상대경로 + '/'
    } else {
        relPath = input.normalize();
        abs = syncHome.resolve(relPath).toAbsolutePath().normalize();
        path = relPath.toString().replace("\\", "/");
    }

    Boolean isDirectory = Files.isDirectory(abs);
    String md5Hex = CMUtil.md5Hex(abs);
    long mtimeSec = Files.getLastModifiedTime(abs).toMillis() / 1000;
    long sizeBytes = Files.size(abs);

    CMFileSyncIndexEntry prevEntry = repo.readEntry(path);
    Boolean isOnline = prevEntry == null ? false : prev.isOnline();

    repo.applyCreateOrModify(path, isDirectory, md5Hex, mtimeSec, sizeBytes, isOnline,
newChangeld);
    writeCursor(userId, deviceId, newChangeld);
    appendChangelog(userId, deviceId, "CHANGEMODE", path, isDirectory, md5Hex, mtimeSec,
sizeBytes, isOnline, newChangeld);
    repo.flushSnapshot();
}

public void applyChangeModeFast(String userId, String deviceId,

```

```

        String path, Boolean isDirectory, String contentHash, long mtimeEpochSec, long
sizeBytes, Boolean isOnline) throws IOException {

        CMFileSyncIndexRepository repo = indexRegistry.getOrLoad(userId, deviceId);
        long newChangeld = repo.lastChangeld() + 1;

        repo.applyCreateOrModify(path, isDirectory, contentHash, mtimeEpochSec, sizeBytes,
isOnline, newChangeld);

        writeCursor(userId, deviceId, newChangeld);

        appendChangelog(userId, deviceId, "CHANGEMODE", path, isDirectory, contentHash,
mtimeEpochSec, sizeBytes, isOnline, newChangeld);

        repo.flushSnapshot();
    }

```

- 멤버 메소드 추가

- 공통 유틸 (cursor / changelog 업데이트)

- 구현 코드

```

private void writeCursor(String userId, String deviceId, long changeld) throws IOException {
    Path cursorFile = Path.of(".cm-settings", "file-sync", "server", userId, deviceId, "cursor");
    Files.createDirectories(cursorFile.getParent());
    Files.writeString(cursorFile, Long.toString(changeld),
        StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING);
}

```

```

private void appendChangeLog(String userId, String deviceId, String op,
    String path, Boolean isDirectory, String hash, long mtime,
    long size, Boolean isOnline, long changeld) throws IOException {
    String today = java.time.LocalDate.now().toString();
    Path logFile = Path.of(".cm-settings", "file-sync", "server",
        userId, "changelog-" + today + ".jsonl");
}

```

```

String logEntry = new ObjectMapper().writeValueAsString(Map.of(
    "changeld", changeld,
    "userName", userId,
    "originDeviceUuid", deviceId,
    "op", op,
    "path", path,
    "isDirectory", isDirectory,
    "contentHash", hash,
    "mtime", mtime,
    "size", size,
    "isOnline", isOnline,
    "tombstone", op.equals("DELETE"),
    "ts", java.time.OffsetDateTime.now().toString()
));

```

```

Files.writeString(logFile, logEntry + System.lineSeparator(),
    StandardOpenOption.CREATE, StandardOpenOption.APPEND);
}

```


CMFILESYNCMANAGER

- 멤버 메소드 수정

public boolean completeFileSync(String initiatorName, UUID initiatorUuid)

- 메소드 역할

- 파일 동기화 세션이 완료되었다는 이벤트를 클라이언트로 전송
- CMFileSyncInfo 내의 클라이언트 파일 목록, generator 정보를 제거하여 동기화 상태 정리

- 수정 방안

- userName 파라미터명을 initiatorName 으로 수정
- sendCompleteFileSync() 메소드에 uuid 파라미터 추가

- 수정 구현

- userName 부분을 initiatorName 으로 변경
...
- Result = sendCompleteFileSync(..) 라인을 다음으로 수정
result = sendCompleteFileSync(initiatorName, initiatorUuid);

- 멤버 메소드 수정

private Boolean sendCompleteFileSync(String initiatorName, UUID initiatorUuid)

- 메소드 역할

- 클라이언트로 동기화 세션 완료 이벤트 전송

- 수정 방안

- userName 파라미터명을 initiatorName 으로 수정
- COMPLETE_FILE_SYNC 이벤트에 cursor 정보 필드 설정 추가
- 위에서 이벤트에 cursor 필드는 추가한 상태

- 수정 구현

- userName 부분을 initiatorName 으로 수정
...
- // get the CMFileSyncGenerator reference 아래 라인을 다음으로 수정
CMFileSyncInfo syncInfo = CMFileSyncInfo().getInstance();
CMFileSyncSenderKey key = new CMFileSyncSenderKey(initiatorName, initiatorUuid);
CMFileSyncGenerator syncGenerator = syncInfo.getSyncGeneratorMap().get(key);
- // create a COMPLETE_FILE_SYNC event 앞에 다음 추가
// device uuid 구하기

```

UUID deviceUuid = syncGenerator.getInitiatorDeviceUuid();
String strDeviceUuid = CMUuidConverter.uuidToString(deviceUuid);
// cursor 구하기
long lastChangeld = syncInfo.getIndexRegistry().getOrLoad(initiatorName,
strDeviceUuid).lastChangeld();

● // create a COMPLETE_FILE_SYNC event 아래 필드 설정 부분 다음으로 수정
int numFilesCompleted = syncGenerator.getNumNewFilesCompleted() +
syncGenerator.getNumUpdateFilesCompleted();

CMFileSyncEventCompleteFileSync fse = new CMFileSyncEventCompleteFileSync();
// sender, receiver, userName 필드 설정 삭제
// 공통 필드 설정 추가
fse.setInitiatorName(initiatorName);
fse.setInitiatorUuid(initiatorUuid);
fse.setInitiatorDeviceUuid(deviceUuid);
// 나머지 필드 설정
fse.setNumFilesCompleted(numFilesCompleted);
fse.setCursor(lastChangeld);

● unicastEvent(..) 라인을 다음으로 수정
return CMEEventManager.unicastEvent(fse, initiatorName, initiatorUuid);

```

- **멤버 메소드 수정**

public Boolean completeNewFileTransfer(String initiatorName, UUID initiatorUuid, Path path)

- **메소드 역할**

- 파일 동기화 수신 측에서 신규 파일 또는 신규 디렉토리 추가 완료에 대한 처리
- Path 는 절대경로
- 현재는 서버지만, 양방향 동기화에서는 클라이언트도 이 메소드 호출 가능

- **수정 방안**

- userName 파라미터명을 initiatorName 으로 수정
- 파라미터로 initiator uuid 추가
- 양방향 동기화 시 내가 서버이거나 클라이언트인 경우에 대한 고려 필요
 - 파라미터명, 메타 파일 업데이트, 이벤트 전송 대상 등
- 파일 추가 완료 시 메타 파일 및 인메모리 정보 업데이트 내용 추가

- **수정 구현**

- userName 부분을 initiatorName 으로 수정
- ...

- `// get CMFileSyncGenerator 까지 기존 코드 동일`
...
- `syncGenerator` 구하는 부분 다음으로 수정
`// CMFileSyncInfo 구하기`
`CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();`
`// generator 구하기`
`CMFileSyncSenderKey key = new CMFileSyncSenderKey(initiatorName, initiatorUuid);`
`CMFileSyncGenerator syncGenerator = syncInfo.getSyncGeneratorMap().get(key);`
 ...
- `// create a COMPLETE_NEW_FILE event 앞에 아래 추가`
...
- 메타 정보 파일 업데이트 (hash, mtime, size, changeld 는 자체 계산)
`// device uuid 구하기`
`// syncGenerator 를 CMFileSyncSenderKey 로 얻도록 이전 부분 수정 필요`
`UUID deviceUuid = generator.getInitiatorDeviceUuid();`
`String strDeviceUuid = CMUuidConverter.uuidToString(deviceUuid);`
`syncInfo.applyCreate(initiatorName, strDeviceUuid, path); // initiatorName, device uuid, path`
- `Return true;`
- 멤버 메소드 수정
`public Boolean completeUpdateFile(String initiatorName, UUID initiatorUuid, Path path)`
 - 메소드 역할
 - 파일 동기화 수신 측에서 파일 업데이트 완료에 대한 처리
 - `Path` 는 상대 경로
 - 현재는 서버지만, 양방향 동기화에서는 클라이언트도 이 메소드 호출 가능
 - 수정 방안
 - `userName` 파라미터명을 `initiatorName` 으로 수정
 - 파라미터로 `initiator uuid` 추가
 - 파일 업데이트 완료 시 메타 파일 및 인메모리 정보 업데이트 내용 추가
 - 수정 구현
 - `userName` 부분을 `initiatorName` 으로 수정
...
 - `// get CMFileSyncGenerator 까지 기존 코드 동일`
...

- syncGenerator 구하는 부분 다음으로 수정


```
// CMFileSyncInfo 구하기
CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
// generator 구하기
CMFileSyncSenderKey key = new CMFileSyncSenderKey(initiatorName, initiatorUuid);
CMFileSyncGenerator syncGenerator = syncInfo.getSyncGeneratorMap().get(key);
...
```
- // create a COMPLETE_UPDATE_FILE event 앞에 다음 추가


```
// device uuid 구하기
UUID deviceUuid = syncGenerator.getInitiatorDeviceUuid();
String strDeviceUuid = CMUuidConverter.uuidToString(deviceUuid);
// 동기화 메타 파일 및 인메모리 정보 업데이트
syncInfo.applyModify(initiatorName, strDeviceUuid, path);
...
```
- 이벤트 필드 설정은 위 다른 부분에서 이미 수정


```
...
```
- unicastEvent() 라인을 다음으로 수정


```
...
Boolean ret = CMEventManager.unicastEvent(fse, initiatorName, initiatorUuid);
...
```
- Return ret;
- 멤버 메소드 추가


```
public Boolean completeDeleteFiles(String initiatorName, UUID initiatorUuid, UUID initiatorDeviceUuid,
List<Path> deletedPathList)
```

 - 메소드 역할
 - 삭제 관련 메타 정보는 CMFileSyncGenerator.deleteFilesAndUpdateBasisFileList()에서 처리
 - 여기서는 삭제 파일 (디렉토리 포함) 리스트와 cursor 정보를 클라이언트로 전송하는 역할
 - 현재는 서버지만, 양방향 동기화에서는 클라이언트도 이 메소드 호출 가능
 - 구현 방안
 - CMFileSyncManager.requestOnlineModeList() 메소드 참고하여 이벤트 설정
 - 메소드 구현


```
// sync info 구하기
CMFileSyncInfo syncInfo = CMFileSyncInfo.getInstance();
// sync generator 구하기
CMFileSyncSenderKey key = new CMFileSyncSenderKey(initiatorName, initiatorUuid);
CMFileSyncGenerator syncGenerator = syncInfo.getSyncGeneratorMap().get(key);
```

```

// 동기화 메타 파일 및 인메모리 정보 업데이트 (각 삭제된 파일에 대해)
String strDeviceUuid = CMUuidConverter.uuidToString(initiatorDeviceUuid);
for( Path path : deletedPathList ) {
    syncInfo.applyDelete(initiatorName, strDeviceUuid, path);
}
// 이벤트 전송 루프 시작
int listIndex = 0;
Boolean sendResult = false;
List<Path> relativeDeletedPathList = toRelativePathList(deletedPathList, getClientSyncHome());
while( listIndex < relativeDeletedPathList ) {
    // 이벤트 객체 생성
    CMFileSyncEventCompleteDeleteFiles fse = new CMFileSyncEventCompleteDeleteFiles();
    // 공통 필드 설정
    fse.setInitiatorName( initiatorName );
    fse.setInitiatorUuid( initiatorUuid );
    fse.setInitiatorDeviceUuid( initiatorDeviceUuid );

    // 이벤트에 넣을 path 서브리스트 구하기
    Path subList = createSubPathListForEvent( fse.getBytesNum(), relativeDeletedPathList, listIndex );
    // update the listIndex
    listIndex += subList.size();
    // set the subList to the event
    fse.setRelativePathList( subList );
    // send the event
    sendResult = CMEventManager.unicastEvent( fse, initiatorName, initiatorUuid );
    if(!sendResult) {
        System.err.println("send error: " + fse);
        return false;
    }
    if( CMInfo._CM_DEBUG ) {
        System.out.println("sent event = " + fse);
    }
}
}

```

CMFILESYNCGENERATOR

(파일 및 디렉토리 삭제 완료 시 메타 파일 적용 코드 추가)

- 멤버 메소드 수정

private void deleteFilesAndUpdateBasisFileList()

- 메소드 역할

- 클라이언트 파일 목록 수신 후에, 서버 파일 목록을 만들고 비교하여 서버에만 존재하는 파일 및 디렉토리를 삭제
 - 남은 파일들에 대해서는 업데이트 진행 상황 추적 준비

- 수정 방안

- 서버에만 존재하는 파일 삭제 후에 해당 파일에 대한 메타 파일 및 인메모리 정보 업데이트
- 서버에만 존재하는 디렉토리 삭제 후에 해당 디렉토리에 대한 메타 파일 및 인메모리 정보 업데이트
- 삭제된 파일 리스트 생성하여 `CMFileSyncManager.completeDeleteFiles()` 호출하여 클라이언트에게 파일 삭제 완료 사실 전달

■ 수정 구현

- `CMFileSyncGenerator` 멤버 변수 `userName` -> `initiatorName` 으로 변경
- 첫 번째 `while()` 문까지 기존 코드 동일
...
- 첫 번째 `while()` 문 이전 라인에 다음 추가

```
// deletedPathList 선언
List<Path> deletedPathList = new ArrayList<>();
// device uuid 구하기
String strDeviceUuid = CMUuidConverter.uuidToString(initiatorDeviceUuid);
```
- 첫 번째 `while` 문 (서버에만 있는 파일 삭제 처리) 내 다음 수정

```
...
// iter.remove() 다음에 아래 구문 추가
// deletedPathList 에 추가
deletedPathList.add( path.subpath(startPathIndex, path.getNameCount()) );
// 동기화 메타 파일 및 인메모리 정보 업데이트
syncInfo.applyDelete(initiatorName, strDeviceUuid, path);
...
```
- 두 번째 `while` 문 (서버에만 있는 디렉토리 삭제 처리) 내 다음 수정

```
...
// iter.remove() 다음에 아래 구문 추가
// deletedPathList 에 추가
deletedPathList.add( path.subpath(startPathIndex, path.getNameCount()) );
// 동기화 메타 파일 및 인메모리 정보 업데이트
syncInfo.applyDelete(initiatorName, strDeviceUuid, path);
...
```
- `If( entryPathList == null )` 블록 전에 다음 추가

```
syncManager.completeDeleteFiles( initiatorName, initiatorUuid, initiatorDeviceUuid,
deletedPathList );
...
```

CMFileSyncEventHandler

- 멤버 메소드 수정
private Boolean processEND_ONLINE_MODE_LIST(CMFileSyncEvent fse)
 - 파일 모드 변경 완료시에는 메타 정보 업데이트 없음
- 멤버 메소드 수정
private Boolean processEND_LOCAL_MODE_LIST(CMFileSyncEvent fse)
 - 파일 모드 변경 완료시에는 메타 정보 업데이트 없음

서버 측 CURSOR 정보 클라이언트로 전달 방안

- 위 메타 정보 업데이트 방안에서 함께 처리

클라이언트 측 메타 정보 파일 관리 방안 설계

- 클라이언트 측 파일 동기화 관련 메타 정보 파일
 - 파일 저장 디렉토리
 - CM 설정 디렉토리 (.cm-settings) → file-sync → client
 - device_uuid
 - 디바이스 uuid
 - 내용
 - 디바이스 고유 식별값 문자열 1줄
 - cursor
 - 로컬 커서
 - 내용
 - 마지막 changeld 숫자 1줄
 - Client-index.json
 - 클라이언트용 인덱스 파일
 - 용도
 - 이 디바이스가 마지막 동기화 완료했을 때의 로컬 파일 상태
 - 서버가 동기화 파일 정보 보냈을 때, 클라이언트 측 파일과 충돌 여부 판별
 - 내용
 - 파일별 서버와의 마지막 동기화 후 수정시간
 - 예시

```
{
  "files": {
    "docs/a.txt": 1700000000,
    "docs/b.txt": 1700000100
  }
}
```

- 클라이언트 측 메타 정보 파일 매핑 정보
 - 파일로부터 읽을 매핑 정보
 - 디바이스 uuid
 - CMFileSyncInfo 내 UUID m_deviceUuid;
 - cursor (마지막 changeld)
 - CMFileSyncInfo 내 String m_strCursor;
 - Client-index (마지막 동기화 후 로컬 스냅샷)
 - CMFileSyncInfo 내 Map<String, Long> m_lastSyncedMtimeMap;
- 클라이언트 측 device uuid 파일 및 매핑 정보 업데이트 방안
 - CMDeviceUuidManager 클래스에서 파일 생성 또는 얻기 메소드 구현
 - CM client 시작할 때 (startCM()) device uuid 파일 생성하거나 기존 파일 내용 읽어서 CMFileSyncInfo 의 deviceUuid 값으로 설정
 - device uuid 파일은 한 번 설정하면 파일을 삭제하지 않는 한 계속 사용하는 것으로 가정하므로, CM 에서 기존 device uuid 파일로 값을 저장하는 경우는 없음
- 클라이언트 측 cursor 파일 및 매핑 정보 업데이트 방안
 - CM client 시작할 때 (startCM()) 파일이 있다면 내용 읽어서 CMFileSyncInfo 의 cursor 값으로 설정 → loadClientCursor()
 - 파일별 동기화 op 완료하면 cursor 인메모리 값만 업데이트 (파일로 저장은 동기화 세션 완료 때)
 - 파일 동기화 세션 전체가 완료하면 cursor 파일 업데이트 (파일 없으면 새로 생성해서 업데이트) -> saveClientCursor()
 - 파일 동기화 세션 완료할 때: COMPLETE_FILE_SYNC 이벤트 수신할 때
 - 서버가 COMPLETE_FILE_SYNC 이벤트에 cursor 정보 필드 추가 필요
- 클라이언트 측 index 파일 및 매핑 정보 업데이트 방안
 - CM client 시작할 때 (startCM()) 파일이 있다면 내용 읽어서 CMFileSyncInfo 의 client-index 값으로 설정 -> loadClientIndex()
 - 파일 동기화 op 나 파일 모드 변경 완료하면 인메모리 index Map 업데이트 -> m_lastSyncedMtimeMap
 - 파일 신규 추가 완료할 때: COMPLETE_NEW_FILE 이벤트 수신할 때, map 에 put (추가 또는 갱신)
 - 파일 업데이트 완료할 때: COMPLETE_UPDATE_FILE 이벤트 수신할 때, map 에 put (추가 또는 갱신)
 - 파일 삭제 완료할 때: COMPLETE_DELETE_FILE 이벤트 수신할 때 (이벤트 추가 필요), map 에서 삭제
 - 로컬 모드 변경 완료할 때: END_LOCAL_MODE_LIST_ACK 이벤트 수신할 때, map 에 put (추가 또는 갱신)
 - 온라인 모드 변경 완료할 때: END_ONLINE_MODE_LIST_ACK 이벤트 수신할 때, map 에 put (추가 또는 갱신)
 - 파일 동기화 세션 전체가 완료하면 index 파일 업데이트 (파일 없으면 새로 생성해서 업데이트) → saveClientIndex()

- 파일 동기화 세션 완료할 때: COMPLETE_FILE_SYNC 이벤트 수신할 때

CMFileSyncInfo

- 수정 방안

- 클라이언트 전용 파일 동기화 `cursor`, `index` 정보 저장을 위한 멤버 변수 추가
- 멤버 변수와 매핑되는 파일 간의 데이터 로드/저장 관리를 위한 메소드 추가

- 수정 구현

```
// CMFileSyncInfo.java (일부 발췌/추가)
// -----
// 필요한 import
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.UUID;
import java.util.Objects;

// ... (기존 클래스 선언부 및 다른 멤버/메소드 생략)

public class CMFileSyncInfo {

    // -----
    // [추가] 클라이언트 측 파일 동기화 식별자/상태
    // -----

    // 파일 동기화 커서 (예: 서버 ChangeLog 상의 마지막 처리 위치 등)
    private long m_lCursor;
    // 클라이언트 베이스 스냅샷: path -> lastSyncedMtime (sec)
    Private final Map<String, Long> m_lastSyncedMtimeMap = new ConcurrentHashMap<>();

    // -----
    // [추가] Getter / Setter
    // -----
    public long getCursor() {
        return m_lCursor;
    }

    public void setCursor(final long lCursor) {
        this.m_lCursor = lCursor;
    }

    Public Map<String, Long> getLastSyncedMtimeMap() {
        Return m_lastSyncedMtimeMap;
    }

    // -----
    // 편의 메소드
    // -----
    Public long getLastSyncedMtime(String relPath) {
        Long mtime = m_lastSyncedMtimeMap.get(relPath);
        long lastSyncedMtime;
```

```

        if( mtime == null ) lastSyncedMtime = -1;
        else lastSyncedMtime = mtime;
        Return lastSyncedMtime;
    }

    Public void setLastSyncedMtime(String relPath, long mtimeSec) {
        M_lastSyncedMtime.put(relPath, mtimeSec);
    }

    Public void removeLastSyncedMtime(String relPath) {
        M_lastSyncedMtimeMap.remove(relPath);
    }

    // -----
    // [추가] 파일 경로 도우미
    // <project_home>/cm-settings/file-sync/client/{device_uuid,cursor}
    // -----
    private Path getClientSyncBaseDir(final String projectHome) {
        return Paths.get(Objects.requireNonNull(projectHome, "projectHome must not be null"),
            ".cm-settings", "file-sync", "client");
    }

    private Path getCursorFile(final String projectHome) {
        return getClientSyncBaseDir(projectHome).resolve("cursor");
    }

    Private Path getClientIndexFile(final String projectHome) {
        Return getClientSyncBaseDir(proejctHome).resolve("client-index.json");
    }

    // -----
    // [추가] 저장 / 로드 API
    // -----

    /**
     * 클라이언트 커서를 파일로 저장합니다.
     * 경로가 없으면 생성합니다.
     *
     * @param projectHome 프로젝트 홈 디렉토리 (예: 앱이 인식하는 루트)
     */
    public void saveClientCursor(final String projectHome) {
        final Path baseDir = getClientSyncBaseDir(projectHome);

        try {
            if (!Files.exists(baseDir)) {
                Files.createDirectories(baseDir);
            }

            // cursor 저장 (null 이면 빈 문자열)
            final Path cursorFile = getCursorFile(projectHome);
            final String cursorStr = (m_lCursor >= 0) ? String.valueOf(m_lCursor) : "";
            Files.writeString(cursorFile, cursorStr);

        } catch (IOException e) {
            System.err.println("CMFileSyncInfo.saveClientCursor(), failed to save: " + e.getMessage());
            e.printStackTrace();
        }
    }
}

```

```

/**
 * 클라이언트 커서를 파일에서 읽어옵니다.
 * 파일이 없으면 해당 필드는 변경하지 않습니다.
 *
 * @param projectHome 프로젝트 홈 디렉토리
 */
public void loadClientCursor(final String projectHome) {
    try {

        // cursor 읽기
        final Path cursorFile = getCursorFile(projectHome);
        if (Files.exists(cursorFile)) {
            final String cursorStr = Files.readString(cursorFile).trim();
            // 비어 있으면 null 로 두어도 되고, 빈 문자열로 두어도 무방
            m_lCursor = cursorStr.isEmpty() ? -1 : Long.valueOf(cursorStr);
        }

    } catch (IOException e) {
        System.err.println("CMFileSyncInfo.loadClientCursor(), failed to load: " + e.getMessage());
        e.printStackTrace();
    }
}

// -----
// 클라이언트 index 파일 로드
// JSON <-> Map 직렬화는 서버에서 사용한 Jackson 이용
// -----

// DTO 는 내부 static class 로 간단히
static class ClientIndexDto {
    public long baseCursor;
    public String generatedAt;
    public Map<String, Long> files;
}

Public void loadClientIndex(String projectHome) {
    Path file = getClientIndexFile(projectHome);
    if (!Files.exists(file)) return;

    try {
        ObjectMapper om = new ObjectMapper();
        ClientIndexDto dto = om.readValue(file.toFile(), ClientIndexDto.class);
        m_lastSyncedMtimeMap.clear();
        if (dto.files != null) {
            m_lastSyncedMtimeMap.putAll(dto.files);
        }
        // dto.baseCursor 있으면 m_lCursor 와 일관성 체크는 선택
    } catch (IOException e) {
        // 로그만 찍고 무시 (손상 시 재생성)
    }
}

// -----
// 클라이언트 index 파일로 저장
// -----
public void saveClientIndex(String projectHome, long baseCursor) {
    Path file = getClientIndexFile(projectHome);

```

```

try {
    Files.createDirectories(file.getParent());
    ObjectMapper om = new ObjectMapper().enable(SerializationFeature.INDENT_OUTPUT);
    ClientIndexDto dto = new ClientIndexDto();
    dto.baseCursor = baseCursor;
    dto.generatedAt = OffsetDateTime.now().toString();
    dto.files = new HashMap<>(m_lastSyncedMtimeMap);
    om.writeValue(file.toFile(), dto);
} catch (IOException e) {
    // 로그만
}
}

```

```

// -----
// [선택] 초기화 헬퍼 (필요 시 호출) -> 초기화는 CMFileSync 생성자에서 하는 것으로 수정
// -----
/**
 * 로컬 파일이 없을 때의 기본 초기값을 적용합니다.
 * - cursor: -1
 */

// ... (기존 CMFileSyncInfo 의 다른 멤버/메소드들)
}
● 멤버 메소드 추가
long currentMtimeSecOrMinusOne(Path abs)
    ■ 메소드 역할
        ● 주어진 절대경로의 마지막 수정 시간 구하기 (파일이 없으면 -1 리턴)
    ■ 구현
        if( !Files.exists(abs) ) return -1L;
        return Files.getLastModifiedTime(abs).toMillis() / 1000;

```

CMCLIENTSTUB

- 메소드 수정
 - public Boolean startCM()**
 - 수정 방안
 - startCM() 마지막 device uuid 값 로드하고 설정한 다음 (CMDeviceUuidManager) cursor, client-index 값 로드 및 설정 진행
 - 수정 구현
 - syncInfo 의 cursor, client-index 로드 메소드 호출 추가 (syncInfo 는 device uuid 설정할 때 이미 사용)
 - ...
 - syncInfo.loadClientCursor(".");
 - syncInfo.loadClientIndex(".");

CMFileSyncEventHandler

- 메소드 수정

processCOMPLETE_NEW_FILE(CMFileSyncEvent fse)

- 수정 방안

- Cursor 인메모리 값 업데이트
- Client-index 인메모리 Map 에 해당 (path, lastSyncedMtime) 설정

- 수정 구현

- Return 전에 다음 추가

```
...
// CMFileSyncManager 구하기
CMInfo cmInfo = CMInfo.getInstance();
CMFileSyncManager syncManager = cmInfo.getServiceManager(CMFileSyncManager.class);

// syncHome 구하기
CMConfigurationInfo confInfo = CMConfigurationInfo.getInstance();
Path syncHome;
if( confInfo.getSystemType().equals("SERVER") ) {
    syncHome = syncManager.getServerSyncHome(fse_cnf.getSenderName());
}
else {
    syncHome = syncManager.getClientSyncHome();
}

// 완료된 path 의 절대 경로 구하기
Path relPath = fse_cnf.getCompletedPath();
Path absPath = syncHome.resolve(relPath).toAbsolutePath().normalize();
// 절대경로의 현재 mtime 구하기
long curMtime = syncInfo.currentMtimeSecOrMinusOne(absPath);

// 인메모리 client-index Map 에 (path, mtime) 추가하기
syncInfo.setLastSyncedTime(relPath, curMtime);

// 인메모리 cursor 값 업데이트
long memCursor = syncInfo.getCursor();
long newCursor = fse_cnf.getCursor();
if( memCursor >= newCursor ) {
    System.err.printf("memory cursor %ld >= received cursor %ld\n", memCursor, newCursor);
}
syncInfo.setCursor(newCursor);
```

- Return true;

- 메소드 수정

private Boolean processCOMPLETE_UPDATE_FILE(CMFileSyncEvent fse)

- 수정 방안

- Cursor 인메모리 값 업데이트
- Client-index 인메모리 Map 에 해당 (path, lastSyncedMtime) 설정
- 위 COMPLETE_NEW_FILE 이벤트 수신 처리 경우와 동일

- 수정 구현

- Return 전에 다음 추가

```
...
// CMFileSyncManager 구하기
CMInfo cmInfo = CMInfo.getInstance();
CMFileSyncManager syncManager = cmInfo.getServiceManager(CMFileSyncManager.class);

// syncHome 구하기
CMConfigurationInfo confInfo = CMConfigurationInfo.getInstance();
Path syncHome;
if( confInfo.getSystemType().equals("SERVER") ) {
    syncHome = syncManager.getServerSyncHome(fse_cuf.getSenderName());
}
else {
    syncHome = syncManager.getClientSyncHome();
}

// 완료된 path 의 절대 경로 구하기
Path relPath = fse_cuf.getCompletedPath();
Path absPath = syncHome.resolve(relPath).toAbsolutePath().normalize();
// 절대경로의 현재 mtime 구하기
long curMtime = syncInfo.currentMtimeSecOrMinusOne(absPath);

// 인메모리 client-index Map 에 (path, mtime) 추가하기
syncInfo.setLastSyncedTime(relPath, curMtime);

// 인메모리 cursor 값 업데이트
long memCursor = syncInfo.getCursor();
long newCursor = fse_cuf.getCursor();
if( memCursor >= newCursor ) {
    System.err.printf("memory cursor %ld >= received cursor %ld\n", memCursor, newCursor);
}
syncInfo.setCursor(newCursor);
```

- Return true;

- 메소드 수정

public Boolean processEvent(CMEvent event)

- 메소드 역할

- 파일 동기화 관련 이벤트 처리 메소드 분기
- 수정 방안
 - COMPLETE_DELETE_FILES 이벤트 처리 메소드 호출 추가
- 수정 구현
 - case CMFileSyncEvent.COMPLETE_UPDATE_FILE -> ... 아래 다음 추가


```
...

case CMFileSyncEvent.COMPLETE_DELETE_FILES -> processResult =
processCOMPLETE_DELETE_FILES(fse);

...
```
- 메소드 추가


```
private Boolean processCOMPLETE_DELETE_FILES(CMFileSyncEvent fse)
```

 - 메소드 역할
 - Cursor 인메모리 값 업데이트
 - 수신한 path 리스트의 각 원소에 대해 client-index 인메모리 Map 에서 해당 (path, lastSyncedMtime) 삭제
 - 구현 내용


```
CMFileSyncEventCompleteDeleteFiles fse_cdf = (CMFileSyncEventCompleteDeleteFiles) fse;

if(CMInfo._DEBUG) {
    System.out.println("=== CMFileSyncEventHandler.processCOMPLETE_DELETE_FILES()
called..");
    System.out.println("fse = " + fse_cdf);
}

// CMFileSyncManager 구하기
CMInfo cmInfo = CMInfo.getInstance();
CMFileSyncManager syncManager = cmInfo.getServiceManager(CMFileSyncManager.class);

// syncHome 구하기
CMConfigurationInfo confInfo = CMConfigurationInfo.getInstance();
Path syncHome;
if( confInfo.getSystemType().equals("SERVER") ) {
    syncHome = syncManager.getServerSyncHome(fse_cdf.getSenderName());
}
else {
    syncHome = syncManager.getClientSyncHome();
}

// 삭제된 path list 의 각 원소에 대해 다음 수행
List<Path> deletedPathList = fse_cdf.getDeletedPathList();
for( Path relPath : deletedPathList ) {
```

```

// 삭제된 path 의 절대 경로 구하기
Path absPath = syncHome.resolve(relPath).toAbsolutePath().normalize();
// 로컬 삭제 상태 확인 (??) -> 필요 없을듯

// 인메모리 client-index Map 에서 (path, mtime) 삭제하기
syncInfo.removeLastSyncedMtime(relPath);
}

// 인메모리 cursor 값 업데이트
long memCursor = syncInfo.getCursor();
long newCursor = fse_cdf.getCursor();
if( memCursor >= newCursor ) {
    System.err.printf("memory cursor %ld >= received cursor %ld\n", memCursor, newCursor);
}
syncInfo.setCursor(newCursor);

return true;

```

- 메소드 수정

private Boolean processCOMPLETE_FILE_SYNC(CMFileSyncEvent fse)

- 메소드 역할

- 동기화 완료 여부를 확인하는 Map 의 원소 개수, 각 경로의 완료 여부를 체크하고 문제가 있으면 false 반환
- 모든 파일 정상 동기화 완료 확인 후, Map 정보 초기화하고 동기화 진행 상태 false 로 변경
- 파일 감시 서비스가 추가 변경을 감지했는지 여부를 확인하고, 그렇다면 새 동기화 시작

- 수정 방안

- Cursor 값 업데이트하고 파일로 저장
 - 메모리 cursor 값을 수신한 이벤트의 cursor 값으로 업데이트
 - 메모리 cursor 값이 파일 cursor 값보다 항상 크거나 같은게 정상인데, 메모리 cursor 가 더 작으면 에러 메시지를 출력하고, 메모리 cursor 값을 파일 cursor 값으로 수정 (파일값 우선 self-heal)
- 인메모리 client-index 정보 (path, lastSyncedMtime) 파일로 저장

- 수정 구현

- // change the file-sync state to stop 앞에 다음 추가
 - ...
 - // 인메모리 cursor 업데이트


```

long memCursor = syncInfo.getCursor();
long newCursor = fse_cfs.getCursor();

```



```

if( memCursor >= newCursor ) {
    System.err.printf("memory cursor %ld >= received cursor %ld\n", memCursor, newCursor);
}
syncInfo.setCursor(newCursor);

// 파일 cursor 가져오기
Path cursorFile = syncInfo.getCursorFile(".");

// cursor 파일 값이 정상이면,
if( cursorFile != null && !cursorFile.isEmpty() ) {
    memCursor = syncInfo.getCursor();
    long fileCursor = Long.valueOf( Files.readString(cursorFile).trim() );
    // memory cursor 가 더 작은 비정상 상태이면,
    if( memCursor < fileCursor ) {
        System.err.printf("cursor regression detected!: mem = %ld, file = %ld\n", memCursor,
fileCursor);
        syncInfo.setCursor(fileCursor);
        // 파일로 메타 정보 저장 필요 없음
    }
    else if( memCursor == fileCursor ) {
        System.out.printf("cursor values are the same: mem = %ld, file = %ld\n", memCursor,
fileCursor);
        // 파일로 메타 정보 저장 필요 없음
    }
    else {
        // 정상적인 memory cursor 가 file cursor 보다 큰 상태이면,
        syncInfo.saveClientCursor(".");
        syncInfo.saveClientIndex(".", syncInfo.getCursor());
    }
}
// cursor 파일이 없거나 비어있으면,
else {
    syncInfo.saveClientCursor(".");
    syncInfo.saveClientIndex(".", syncInfo.getCursor());
}
...

```

- 멤버 메소드 수정

private Boolean processEND_ONLINE_MODE_LIST_ACK(CMFileSyncEvent fse)

- 파일 모드 변경시에는 메타 정보 업데이트 없음

- 멤버 메소드 수정

private Boolean processEND_LOCAL_MODE_LIST_ACK(CMFileSyncEvent fse)

- 파일 모드 변경시에는 메타 정보 업데이트 없음